Imperial College London

MEng Individual Project

Imperial College London

Department of Computing

Data Poisoning Attacks against Adaptive (Continual) Learning

Author:
Dillan Scott

Supervisor:
Prof. Emil Lupu

Second Marker:
Prof. Sergio Maffeis

June 12, 2026

Abstract

Adaptive intrusion-detection systems pair a drift detector with a base learner that retrains whenever the detector identifies a shift in the incoming data's distribution. The Contrastive NCM detector of Kuppa and Le-Khac employs a contrastive autoencoder and a Nearest Class Mean classifier and is reported to be adversarially robust. That robustness, however, has only ever been evaluated for the detector in isolation. We evaluate it as it would be deployed: coupled to a downstream classifier whose retraining it triggers.

We find that the coupling introduces exploitable surfaces, and that the adversary's leverage is over the system's adaptation rather than its decision boundary. An aimed feature-space mean-shift forces an erroneous concept-discovery event both white-box and grey-box, retaining roughly 70% of its effect against a detector trained on disjoint data. It transfers because it targets the aggregate signal the detector uses to decide retraining rather than the per-sample flags the detector is built on. Averaging over a batch cancels the per-sample noise that otherwise stops a perturbation crafted against one encoder from carrying to another. A firing gate added to stop premature and forced retraining is double-edged: an ablation proves that any constraint on the retraining buffer's makeup that resists this forcing simultaneously lets an attacker suppress adaptation by dilution, starving the system of the updates it needs; the same dilution, with no adversary at all, prevents detection of a genuine novel class that the detector's own false positives hold below the gate's threshold.

Coupling a robust detector to a base learner does not necessarily mean the system will inherit the detector's robustness; it creates new attack surfaces that a detector-only evaluation simply cannot see.

Contents

1 Introduction **4**
- 1.1 Context and Motivations . . . 4
- 1.2 Scope . . . 5
- 1.3 Objectives . . . 5
- 1.4 Research Questions . . . 5
- 1.5 Thesis Structure . . . 5

2 Background **7**
- 2.1 Concept Drift . . . 7
  - 2.1.1 Taxonomy of Drift . . . 7
- 2.2 Concept Drift Detection . . . 8
  - 2.2.1 General Architecture of Drift Detection Systems . . . 8
  - 2.2.2 Drift Detection Algorithms . . . 9
  - 2.2.3 Limitations and Weaknesses . . . 10
- 2.3 Adversarial Concept Drift . . . 11
  - 2.3.1 Adversarial Objectives . . . 11
  - 2.3.2 Instance-Based Poisoning . . . 12
  - 2.3.3 Concept-Based Poisoning . . . 12
- 2.4 Class-Incremental Learning and Exemplar Memory . . . 12
- 2.5 Adversarially Robust Solutions . . . 13
  - 2.5.1 Robust Restricted Boltzmann Machine for Drift Detection . . . 13
  - 2.5.2 The Contrastive NCM Detector . . . 14
- 2.6 Summary . . . 15

3 Target System and Threat Model **17**
- 3.1 Target System Architecture . . . 17
  - 3.1.1 The Base Learner . . . 17
  - 3.1.2 The Drift Detector . . . 17
  - 3.1.3 System Integration and Adaptation Cycle . . . 17
- 3.2 Threat Model . . . 18
  - 3.2.1 Attacker Objectives . . . 19
  - 3.2.2 Attacker Knowledge . . . 19
  - 3.2.3 Attacker Capabilities . . . 19
  - 3.2.4 Adopted Threat Model . . . 19
- 3.3 Summary . . . 20

4 Attack Taxonomy and Objectives **21**
- 4.1 The Attack Surface of the Coupling Layer . . . 21
- 4.2 Attacks on the Trigger Channel . . . 22
  - 4.2.1 Forcing Adaptation (False Positives) . . . 22
  - 4.2.2 Suppressing Adaptation (False Negatives) . . . 22
- 4.3 Attacks on the Composition Channel . . . 23
  - 4.3.1 Poisoning $\mathcal{M}$ . . . 23
  - 4.3.2 Backdooring $\mathcal{M}$ . . . 23
  - 4.3.3 Poisoning and Backdooring $\mathcal{D}$ . . . 23
- 4.4 Attack Persistence and Maintenance . . . 23

4.5 Compound Attacks . . . 24
4.6 Summary . . . 24

5 Implementation **26**
5.1 Datasets and Preprocessing . . . 26
5.2 Reimplementing and Reproducing the Contrastive NCM Detector . . . 27
5.2.1 Reimplementation . . . 27
5.2.2 Reproduction . . . 27
5.3 The Coupled Adaptive System . . . 28
5.3.1 Architecture and Streaming Harness . . . 28
5.3.2 Three System Configurations . . . 28
5.4 Completing the underspecified Components . . . 29
5.4.1 The Count-and-Fraction Firing Gate . . . 29
5.4.2 Threshold Calibration . . . 29
5.4.3 Retrain-Set Composition and Encoder Stability . . . 29
5.4.4 Evaluation Surface . . . 30
5.5 Attack Implementation . . . 30
5.5.1 The Injection Model . . . 31
5.5.2 Suppressing Adaptation . . . 31
5.5.3 Forcing Adaptation . . . 31
5.5.4 Poisoning the Base Learner . . . 32
5.5.5 Backdooring the Base Learner . . . 32
5.6 Summary . . . 32

6 Evaluation **34**
6.1 Experimental Settings . . . 34
6.2 Baseline Behaviour Under Genuine Drift . . . 34
6.3 RQ1 – The Attack Surfaces of the Coupling . . . 36
6.4 RQ2 – The Composition Channel: Poisoning and Backdooring the Base Learner . . . 37
6.5 RQ3 – The Trigger Channel: Forcing and Suppression Trade-Offs . . . 39
6.5.1 Forcing Per-Sample Detection . . . 39
6.5.2 Forcing the Concept Trigger . . . 39
6.5.3 Decomposing the Firing Gate . . . 40
6.6 Summary . . . 42

7 Conclusion **43**
7.1 Future Work . . . 44
7.2 Limitations . . . 44

8 Declarations **47**
8.1 Use of Generative AI . . . 47
8.2 Ethical Considerations . . . 47
8.3 Sustainability . . . 47
8.4 Availability of Data and Materials . . . 48

Chapter 1

Introduction

1.1 Context and Motivations

It is becoming increasingly necessary for machine learning systems to operate in dynamic environments where data arrives as a stream with no guarantees about its underlying distribution. Examples include fraud detection, network intrusion detection, and content moderation, where new observations are generated sequentially, and historical data may eventually become outdated.

Learning under these conditions is referred to as adaptive or continual learning. In contrast to traditional batch learning, where models are static post-deployment, adaptive models are repeatedly updated. The motivation for these online updates is not only to handle the volume or velocity of streaming data, but also to account for concept drift (changes in the joint distribution of inputs and outputs over time). The performance of a static model, which is accurate on current data, may degrade as the environment changes. Consequently, mechanisms known as drift detectors are used to monitor incoming data streams and signal the need for model updates when necessary.

While drift detection has been extensively studied in non-adversarial settings [1, 2, 3], critical security concerns arise when operating in environments where strategic or malicious actors may influence data streams. In domains such as fraud detection and cybersecurity, adversaries may deliberately manipulate data to evade detection, degrade model performance, or force harmful model adaptations. Unlike poisoning attacks against fixed models, adversarial attacks against adaptive systems can aim to exploit the update mechanism, using it as a vector to inject malicious concepts, suppress valid adaptations, or corrupt the detector's learned representation.

In response to this threat, recent research has proposed adversarially robust drift detectors. The Contrastive Nearest Class Mean (NCM) detector of Kuppa and Le-Khac [4], the target system of this work, uses a contrastive loss to map inputs into a latent space where same-class samples cluster tightly, classifies incoming samples by their distance to learned class prototypes using a combined Euclidean and Riemannian metric designed to reject adversarial subspaces, and grows the prototype set automatically when the accumulated displacement of drifted embeddings crosses a threshold. Another approach by Korycki and Krawczyk [5] pursues similar goals through reconstruction-error monitoring with an energy-based model, specifically a Robust Restricted Boltzmann Machine. Both have demonstrated robustness against certain poisoning strategies.

However, both works share a fundamental limitation when viewed from the perspective of a fully coupled adaptive learning pipeline. Kuppa and Le-Khac [4] evaluate their detector in isolation: their experiments measure the detector's ability to correctly classify incoming samples as drifted or not, reporting precision, recall and F1 against held-out drifted classes. No downstream base learner is used, meaning their robustness claims speak only to the detector's own classification performance and say nothing about whether those properties extend to protecting a separate model that depends on the detector to govern its adaptation. Korycki and Krawczyk [5] do include a base learner, but they choose a model that updates continuously, instance by instance, regardless of drift signals, meaning the detector is advisory rather than controlling.

The coupling between detector and base learner is itself an attack surface that no detector-only

evaluation can characterise, and the architectural details of that coupling (when a downstream classifier retrains relative to detector signals) materially determine which attacks succeed.

1.2 Scope

We examine adversarial attacks against an adaptive learning pipeline consisting of a static MLP classifier coupled with the Contrastive NCM detector [4]. The work is centred on understanding whether this state-of-the-art robust detector is sufficient to protect a trigger-dependent base learner when the two components are deployed together as a coupled system.

We take an adversarial perspective, proposing attacks against the coupled system that target each element both individually and in combination. We then attack the surfaces identified by this taxonomy to characterise exactly where the system is vulnerable.

We do not refute the robustness claims of Kuppa and Le-Khac [4]; rather, we test whether those claims extend to the protection of a downstream base learner in a realistic adaptive learning architecture. In doing so, we clarify the practical security guarantees offered by the detector and identify the conditions under which the coupled system remains vulnerable.

1.3 Objectives

The primary objective of this research is to characterise how an adversary can corrupt a coupled adaptive learning system equipped with the Contrastive NCM detector, and at what budget such corruption can be sustained.

We meet this objective in three parts. First, we formalise a threat model and attack taxonomy for the coupled system, identifying attack surfaces that a trigger-dependent architecture exposes but a continuous-update architecture does not. Second, we attack these surfaces to determine whether the base learner can be corrupted. Third, we attack the detector itself to determine whether it can be manipulated to control the base learner's adaptation.

1.4 Research Questions

To meet the objectives described in the previous section, we address the following research questions:

1. What attack surfaces does trigger-gated retraining – coupling a drift detector and an otherwise static base learner – expose, that a continuously updated learner does not?
2. Can an adversary corrupt the base learner through the retraining dataset, and what corruption does the coupled system resist or allow?
3. Can an adversary control the firing trigger to either force false positives or suppress true positives, and what does a gating mechanism hardened against one failure mode cost in the other?

None of these has an obvious answer. RQ1 cannot be settled by studying the detector alone: the surfaces a trigger-gated architecture exposes are precisely the ones a detector-only evaluation is structurally blind to, so answering it at all requires reconstructing and attacking the deployed, coupled system. RQ2 is genuinely open because a base learner rebuilt from scratch on a class-balanced retraining set may simply outvote a minority of poisoned flows; whether the composition channel can corrupt such a learner, and by what kind of payload, cannot be assumed in advance. And RQ3 admits no trivial answer: forcing and suppression pull the firing gate's design in opposite directions, so any floor added to resist one failure mode threatens to open the other. Characterising that trade-off, rather than simply hardening the gate, is where the difficulty lies.

1.5 Thesis Structure

The remainder of this thesis is organised as follows:

- Chapter 2 reviews the background literature on continual learning and adversarial attacks.
- Chapter 3 formalises the target system and establishes the threat model, defining the adversary's objectives, knowledge, and capabilities.
- Chapter 4 presents a taxonomy of attacks against the coupled system and introduces the idea of maintenance cost.
- Chapter 5 describes the implementation of the coupled system, including four architectural refinements that clarify the underspecified components of the paper's intended architecture, and attacks selected for evaluation.
- Chapter 6 evaluates the implemented attacks against the coupled system, measuring their impact on the base learner's predictive performance and the detector's performance.
- Chapter 7 summarises the findings, discusses their implications for the design of robust adaptive learning systems, and outlines directions for future research.
- Chapter 8 provides a discussion of wider ethical, legal, professional and societal issues surrounding this project and the accompanying research.

Chapter 2

Background

This chapter establishes the theoretical foundations and reviews the relevant literature to contextualise the later contents of this thesis. It begins by formally defining concept drift and outlining its taxonomy based on probabilistic decomposition and temporal characteristics. Subsequently, the general architecture of drift detection systems is analysed, alongside a review of some existing concept drift detectors and their inherent limitations. The chapter then moves on to consider a security perspective, introducing the notion of adversarial concept drift, where distributional shifts are manipulated by potentially malicious actors. Before turning to defences, it introduces the foundations of class-incremental learning and exemplar-based memory, which underpin how adaptive learners update without forgetting. Finally, the chapter concludes by discussing the existing robust solutions, which have been proposed to mitigate these challenges.

2.1 Concept Drift

Concept drift generally refers to changes in the statistical relationship between input features and target labels, characterised as a change in the true joint distribution $p(X, y)$ [1]. Formally, concept drift between time points t_0 and t_1 is said to occur when

$$p_{t_0}(X, y) \neq p_{t_1}(X, y), \tag{2.1}$$

where p_{t_0} and p_{t_1} denote the joint distributions of the input variables X and target variable y at times t_0 and t_1, respectively [2].

2.1.1 Taxonomy of Drift

Probabilistic Nature

The joint probability can be given by the decomposition

$$p(X, y) = p(y \mid X)p(X) \tag{2.2}$$

where $p(y \mid X)$ is the conditional distribution of the target given the input features and $p(X)$ is the probability distribution of the input features. Based on this, the literature [1, 2, 5] distinguishes between two forms of concept drift:

1. *Real drift*: When $p(y \mid X)$ changes without a change in $p(X)$ (Conditional Change) or with it (Dual Change). This is the more critical form of drift, as it directly affects decision rules or classification boundaries, making model adaptation necessary.
2. *Virtual drift (feature change)*: When $p(X)$ changes without affecting $p(y \mid X)$. While this may appear less important than real drift because the decision function is left unchanged, it cannot be ignored. In particular, changes in $p(X)$ may trigger adaptation or forgetting mechanisms, potentially leading to unnecessary or even harmful model updates.

Temporal Characteristics

In addition to the above types, based on probabilities, concept drift can be categorised based on how the change occurs over time. Namely, four common patterns are shown in Fig. 2.1 [6].

1. *Sudden/abrupt drift* is when the instance distribution or concept changes within a very short time frame. The previous concept immediately becomes invalid, and the learner must adapt quickly to maintain performance [5].
2. *Gradual drift* is where instances from the old and new concepts coexist for a period, with old appearing with decreasing frequency [5].
3. *Incremental drift* is when there is a smooth and continuous progression from one concept to another through a series of intermediate concepts [2, 5].
4. *Recurring concepts* refers to when previously seen concepts reappear after some time, such as in seasonal or cyclical patterns [2]. They may reappear either suddenly or incrementally [7].

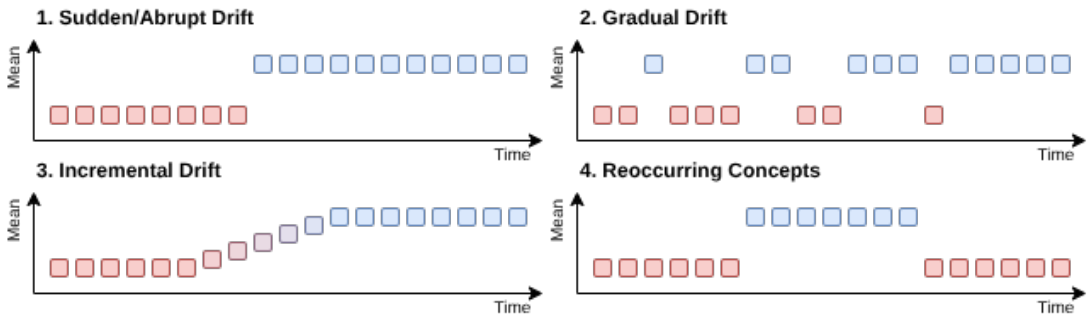

Figure 2.1: Examples of concept drift types based on temporal patterns

2.2 Concept Drift Detection

In data stream learning, concept drift is typically addressed by augmenting the predictive model (the base learner) with a drift detection mechanism. In general, drift detectors attempt to identify changes in the data distribution by analysing the base learner's behaviour [3]. Rather than continuously retraining the learner, drift detectors act as a supervisory mechanism, triggering an adaptation process, such as model retraining, parameter update, or selective forgetting of historical data.

2.2.1 General Architecture of Drift Detection Systems

Although they are algorithmically diverse, most concept drift detection systems follow a common architectural pattern. This architecture can be decomposed into four conceptual components:

1. *Observation source*: The detector monitors a stream of signals derived either from the data itself or from the performance of a base learner. Common sources include error rates, loss values, feature statistics, or latent representations.
2. *Summary mechanism*: Observations are aggregated over time using sliding windows, cumulative statistics, or incremental estimators. This step compresses the raw stream into a tractable representation that reflects the system behaviour of a given period.
3. *Change test*: A statistical test, threshold comparison, or learned decision rule evaluates whether the observed summary deviates significantly from a reference/historical summary. This comparison may rely on bounds derived from concentration inequalities, hypothesis tests, or reconstruction error metrics.
4. *Decision and response*: Upon detecting significant deviation, the detector emits a signal (sometimes distinguishing between a warning and confirmed drift) which triggers downstream actions such as retraining, window resetting, or model replacement.

This modular structure highlights an important property of drift detectors: they act as *control mechanisms* governing when and how adaptive learning occurs. As such, their behaviour has long-term consequences for system performance, stability, and robustness.

2.2.2 Drift Detection Algorithms

While the general architecture remains consistent, specific implementations differ in their observation sources and statistical tests. The literature typically categorises these methods into error rate-based and data distribution-based approaches [6].

Error Rate-Based Detectors

These methods monitor the online performance of the base learner (e.g. classification accuracy). They function based on the assumption that a statistically significant increase or decrease in model performance implies a change in the underlying data distribution [6]. Drift detectors in this category include:

- *Drift Detection Method (DDM)* [8]: This method models the number of errors as a random variable from a Binomial distribution. It tracks the online error rate p_t and its standard deviation $s_t = \sqrt{p_t(1-p_t)/t}$ at time t. The algorithm maintains the minimum values observed so far, p_{min} and s_{min}, updating them whenever $p_t + s_t < p_{min} + s_{min}$. A warning level is triggered if:

$$p_t + s_t \geq p_{min} + 2s_{min}, \tag{2.3}$$

  at which point incoming examples are buffered while using the old learner for predictions. If the error rate continues to rise and drift is confirmed via the condition:

$$p_t + s_t \geq p_{min} + 3s_{min}, \tag{2.4}$$

  a new learner is induced using the buffered examples, and the old learner is discarded.

- *ADaptive WINdowing (ADWIN)* [9]: This method uses a sliding window W that dynamically adjusts its size, based on the rate of change in the data stream. It operates by examining all possible cuts of the current window into two sub-windows, W_0 and W_1, representing historical and newer data, respectively. Drift is detected if the absolute difference between the averages of these sub-windows is statistically significant:

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| \geq \epsilon_{cut} \tag{2.5}$$

  where ϵ_{cut} is an adaptive threshold (determined using Hoeffding bounds and a confidence interval δ). When this threshold is reached, the historical data (W_0) is discarded as the two sub-windows are seen to have come from different data distributions. The authors also proved that the false positive and false negative rates are bounded by δ.

Data Distribution-Based Detectors

In contrast, these methods monitor the statistical properties of the input features ($p(X)$), independent of the base learner's behaviour and error rate. They rely on the assumption that drift can be signalled by a statistically significant deviation in the distribution of new data when compared to a historical reference window [6]. Because they analyse the input data itself, these methods can detect virtual drift that does not immediately alter the decision boundary and can also function when ground truth labels are unavailable. Examples of data distribution-based detectors include:

- *Statistical Change Detection for multidimensional data (SCD)* [10]: This detector addresses the challenge of multidimensional distribution change by employing a non-parametric statistical test known as the Density Test. The baseline/historical window S is partitioned into two halves: S_1 for modelling and S_2 for testing. It utilises a Kernel Density Estimator (KDE) on S_1, to get the density function $\mathscr{K}_{S_1}$. The test statistic δ is then defined using the difference in log-probability density of $\mathscr{K}_{S_1}$ at S' and at S_2. Specifically:

$$\delta = LLH(\mathscr{K}_{S_1}, S') - \frac{|S'|}{|S_2|} \times LLH(\mathscr{K}_{S_1}, S_2) \tag{2.6}$$

where LLH is the log-likelihood. Drift is confirmed if δ falls below a critical value C_{max} (algorithmically derived from a user-defined significance level p).

- *Principal Component Analysis-Based Change Detection (PCA-CD)* [11]: This method works on multidimensional data streams by projecting the data onto lower-dimensional spaces using PCA. It extracts the top k principal components that explain a substantial portion of the variance from the historical window and then monitors new data projected onto these components as separate univariate streams. Similarly to SCD, the density of these projections is estimated using KDE, for example. A divergence metric, such as the intersection area under the curves of the reference ($\hat{f}$) and test ($\hat{g}$) distributions:

$$D_A\left(\hat{g} \,||\, \hat{f}\right) = 1 - \int_x \min\left(\hat{f}(x), \hat{g}(x)\right) dx \tag{2.7}$$

is calculated for each of the k components. Then the final change score is taken as the maximum of these. To detect drift, the method applies the Page-Hinkley (PH) test with a dynamic threshold to the aggregate score, triggering an alarm when the cumulative deviation from the historical mean exceeds a given limit.

2.2.3 Limitations and Weaknesses

While the drift detectors described above do provide robust mechanisms for adaptation, they are subject to inherent limitations and trade-offs between sensitivity (minimising false negatives) and specificity (minimising false positives). Rather than arising from flawed algorithms, many of these weaknesses are due to the statistical assumptions made by classical drift detection; most notably, finite-sample estimation, approximate stationarity within windows, and the use of fixed confidence bounds to identify change.

These limitations can be mapped onto the general architecture of drift detectors introduced earlier in Section 2.2.1. Choices made at the level of the observation source (e.g. error rates or feature statistics), the summary mechanism (e.g. windowing or cumulative statistics), and the change test (e.g. hypothesis tests or thresholds) impose constraints on what types of drift can be reliably identified and how quickly.

Sensitivity to Gradual Drift

A limitation of some error-based detectors, such as DDM, is the reduced ability to distinguish between stationary noise and gradual concept drift. In the presence of abrupt drift, the error rate typically exhibits a sharp increase, triggering the detector's thresholds (defined for DDM in Eq. (2.3) and Eq. (2.4)). However, during a gradual drift, the error rate may increase too slowly to violate the statistical bounds.

Under such conditions, the detector may update its internal reference statistics (p_{min} and s_{min} in the case of DDM), effectively incorporating the drifted distribution into its baseline instead of identifying it. While adaptive windowing approaches, such as ADWIN, mitigate this by dynamically adjusting the window size, they are still bounded by the resolution of their sliding window. Drift, which remains below the statistical bound (defined for ADWIN as ϵ_{cut} in Eq. (2.5)), will remain undetected until enough samples accumulate to reach statistical significance.

Fundamentally, this limitation highlights the difficulty in distinguishing between slow drift and stationary noise under bounded confidence levels, without making additional assumptions about the drift process. Any attempt to increase sensitivity by lowering detection thresholds inevitably raises the false positive rate, highlighting a trade-off key to all hypothesis test-based or threshold-based drift detectors.

Verification Latency and Label Availability

Error-based detectors suffer from a critical dependency on ground truth availability. They assume that labels are available immediately or with minimal delay after the prediction is made, to compute prediction errors and monitor performance degradation. In many real-world streaming applications, such as credit fraud detection or loan forecasting, the true label may arrive with significant delay (verification latency) or may never arrive at all.

During periods of high verification latency, the detector effectively cannot observe its primary signal, leaving the system blind to changes in performance. If drift occurs during such a period, the system will continue to trust an outdated model, accumulating loss until detection becomes possible.

Virtual Drift and False Positives

Distribution-based detectors address the problem with label dependency by monitoring the input feature distribution $p(X)$ rather than model error. While this enables detection in settings with delayed or missing labels, it introduces a different weakness related to virtual drift. A statistically significant change in $p(X)$ does not always imply a change in the conditional distribution $p(y \mid X)$. It therefore does not guarantee that the predictive decision boundary has changed.

Consequently, distribution-based detectors may generate false positives, triggering model retraining or data forgetting even when performance remains stable (there is no real drift). Such unnecessary adaptation can be computationally expensive and may temporarily degrade performance.

Dimensionality and Feature Relevance

In high-dimensional settings, distribution-based detectors rely on density estimation or projection techniques that are sensitive to the curse of dimensionality. As the number of dimensions increases, data sparsity reduces the reliability of density estimates, while dimensionality reduction risks discarding directions along which drift is relevant. Moreover, distribution-based detectors typically treat all features with equal importance, meaning they are susceptible to changes in weakly relevant features that may not materially affect model performance.

2.3 Adversarial Concept Drift

Classical concept drift assumes that changes in the data distribution arise from natural, exogenous factors such as evolving user behaviour, seasonality, or changes in the underlying environment. In contrast, adversarial drift assumes that there is a purposeful and potentially malicious actor attempting to manipulate our adaptive learning system [5]. The potential goals they may have will be discussed in the next section.

In such settings, the drift process is no longer passive. Instead, the data stream itself becomes an attack surface, and the drift detector becomes a target. Adversarial drift extends traditional data poisoning attacks by exploiting temporal dynamics and specific detector behaviours. Rather than corrupting a static training set, the adversary seeks to control when adaptation is triggered, what data is included during retraining, and which information is forgotten. This introduces a new class of threats that cannot be fully explained by classical adversarial learning models alone [12].

2.3.1 Adversarial Objectives

An adversary interacting with an adaptive learning system may pursue several objectives, which are not mutually exclusive.

A primary objective could be *performance degradation*, where the adversary seeks to reduce the predictive accuracy or increase the loss of a system [13]. Unlike poisoning attacks, adversarial drift may be gradual, ensuring that degradation accumulates slowly while remaining below detection thresholds.

This could be seen as a potential secondary objective: *evasion of detection*. Here, the adversary aims to avoid the activation of the drift detector while pursuing their primary objective. This could involve crafting inputs that move the decision boundary while preserving the statistics monitored, hence exploiting the blind spots of classical detectors.

Alternatively, an adversary may attempt *forced adaptation*, deliberately triggering false drift alarms to cause unnecessary retraining or forgetting. Such behaviour can destabilise the learning process, increase computational costs, or induce catastrophic forgetting of valid historical concepts [5]. Conversely, an adversary could seek to *block adaptation*, injecting contradictory data points during a period of valid concept drift, in an attempt to nullify the adaptation process [5].

Finally, adversaries may seek *control over the learned concept*, gradually forcing the system to learn a decision boundary favourable to the attacker's goals. In this case, the detector is actively exploited as a mechanism for injecting the poisoned instances during adaptation cycles.

2.3.2 Instance-Based Poisoning

Instance-based poisoning attacks are those in which an adversary injects or modifies individual data points in the stream to influence the base learner or drift detector. These attacks are powerful because the poisoned instances can be distributed over time, allowing adversaries to exploit both the learner's update behaviours and the detector's temporal assumptions. For example, poisoned samples may be injected intermittently or gradually, ensuring that individual instances appear benign while the cumulative effect leads to performance degradation or detector manipulation.

From a drift detection perspective, instance-based poisoning can be used to:

- *Bias summary statistics*: By subtly increasing error rates or similar statistics within certain bounds, the adversary can influence the detector's inner mechanisms.
- *Trigger false alarms*: Bursts of or extreme poisoned instances may surpass thresholds, triggering unnecessary adaptation or window resets.
- *Suppress true drift detection*: Conversely, carefully engineered samples can stabilise monitored statistics during periods of genuine drift, delaying or preventing detection [5].

Crucially, even small, strategically placed perturbations can be sufficient to accumulate and have the desired effect, particularly against detectors based on hypothesis testing or confidence bounds. This makes instance-based poisoning difficult to distinguish from natural noise or gradual drift, especially in high-variance data streams.

2.3.3 Concept-Based Poisoning

Concept-based attacks function at a higher level of abstraction than instance-based attacks. Rather than focusing on individual samples, the adversary aims to directly mimic a shift in the data-generating process to falsify genuine concept drift. The adversary engineers the drift trajectory to achieve a desired outcome, such as the misclassification of specific regions of the input space or the biased treatment of certain inputs.

By anticipating how the detector responds to changes in monitored statistics, the adversary can:

- *Force adaptation*: Triggering adaptation where the system interprets the adversarial concept as a genuine distributional change [5].
- *Exploit forgetting mechanisms*: Causing the system to discard historically valid or important data.
- *Suppress adaptation*: Balancing genuine concept drift with a conflicting distribution, preventing the detector from reaching its change threshold or negatively impacting the adaptation process.

2.4 Class-Incremental Learning and Exemplar Memory

The drift detectors described so far are concerned with deciding when to adapt. Equally important is how a model is updated once said adaptation is triggered. When a learner is only retrained on newly arrived data, it can overwrite previously learned knowledge, a phenomenon known as catastrophic forgetting [14]. This is especially problematic in streaming settings where recurring concepts are common and may reappear after the model has specialised to more recent traffic.

A common countermeasure is rehearsal (or replay), where a small memory of representative past examples, called exemplars, is retained and used alongside new data during each update [15]. Given a finite memory budget, an important question is which examples to keep so the retained subset remains faithful to the data it summarises.

A widely adopted solution is the iCaRL (Incremental Classifier and Representation Learning) mechanism introduced by Rebuffi et al. [16], which classifies using a nearest-mean-of-exemplars rule. Given a class with feature embedding $\phi(\cdot)$ and true mean

$$\mu = \frac{1}{n} \sum_{x} \phi(x), \tag{2.8}$$

herding greedily builds an ordered exemplar set $\{p_1, \ldots, p_m\}$ of target size m, selecting the k^{th} exemplar as the sample that brings the running exemplar mean closest to μ:

$$p_k = \arg\min_{x} \left\| \mu - \frac{1}{k} \left(\phi(x) + \sum_{j=1}^{k-1} \phi(p_j) \right) \right\|. \tag{2.9}$$

Because each successive exemplar maximally reduces the difference between the stored mean and the true class mean, any prefix of the resulting set remains representative of the class centroid, and the budget m can be adjusted without re-selecting from scratch.

This property is particularly valuable for nearest-mean classifiers operating under bounded memory. Since the class prototype is computed from the stored exemplars rather than the full data history, a herded memory preserves the representation of that prototype while bounding both storage and retraining cost. This makes it a natural memory-management policy for an NCM-based drift detector that must register newly discovered concepts and retain a limited, representative sample of each for future retraining, a point we return to when describing the drift detector adopted in this thesis (Section 2.5.2).

2.5 Adversarially Robust Solutions

To address the vulnerabilities of traditional detectors, recent research proposes methods explicitly designed to withstand adversarial manipulation. These solutions employ techniques like contrastive embedding and generative modelling to differentiate between natural and adversarial distributional shifts.

2.5.1 Robust Restricted Boltzmann Machine for Drift Detection

Korycki and Krawczyk [5] introduce the Robust Restricted Boltzmann Machine for Drift Detection (RRBM-DD), a generative neural network that detects drift by monitoring the reconstruction error of incoming samples against a learned model of the past data distribution. That is, the RRBM encodes the recent distribution directly in its weights and the detector treats any sample that the model fails to reconstruct accurately as evidence of distributional change. The architecture also incorporates two distinct mechanisms designed to remain robust when the incoming stream is adversarially manipulated rather than naturally drifting.

To counter instance-based poisoning, the model is trained with a robust gradient descent procedure using M-estimators that softly truncate the influence of samples whose loss exceeds a learned tolerance. To counter concept-based poisoning, the model's energy function is augmented with a gating mechanism over the hidden layer and a Gaussian noise model over the visible layer. The gating allows individual hidden units to be deactivated when their contribution is dominated by an adversarial cluster, and the noise model damps reconstruction-error spikes caused by adversarial perturbations.

Korycki and Krawczyk evaluate the RRBM-DD as part of a coupled adaptive system where the downstream base learner is updated continuously. They employ an adaptive Hoeffding tree that incrementally revises its splits with each new labelled sample, independently of any drift signal. In this configuration, the detector's role is advisory: it can declare that drift has occurred, causing a full retraining, but the model still adapts continuously and so does not depend entirely on the detector to trigger updates. This contrasts with the setting examined in this thesis, in which a static base learner updates only when explicitly triggered by the detector and therefore inherits both the detector's robustness properties and its decision latency. The implications of this difference for adversarial robustness are taken up in Chapter 3.

2.5.2 The Contrastive NCM Detector

Kuppa and Le-Khac [4] introduce the Contrastive NCM detector, composed of three key components. A contrastive autoencoder which maps each raw input vector into a lower-dimensional latent space where same-class samples cluster tightly. A Nearest Class Mean (NCM) classifier which uses a hybrid Euclidean-Riemannian distance metric to decide whether the incoming sample is consistent with one of the known classes or should be flagged as drifted. Finally, a recursive concept-discovery mechanism which accumulates drifted embeddings over time and, once a threshold is exceeded, registers a new class prototype or concept. This combination is designed to remain robust under adversarial inputs by combining the manifold-tightening effect of contrastive training with a distance metric that distinguishes adversarial samples from genuine novel ones.

Contrastive Autoencoder

The encoder $h : \mathbb{R}^d \to \mathbb{R}^r$ maps each d-dimensional input x to a lower-dimensional latent embedding $h(x)$ (with $r < d$); we write $h_i = h(x_i)$ for the embedding of sample x_i. The autoencoder is trained jointly under two losses. The first is the standard mean-squared reconstruction loss

$$\mathcal{L}_{\text{MSE}} = \|g(h(x)) - x\|_2^2, \tag{2.10}$$

where $g(\cdot)$ is the decoder. The second is a supervised contrastive term which aims to make the positive pairs attracted and the negative samples separated

$$\mathcal{L}_{\text{contrastive}} = -\sum_{i=1}^{B} \log \left[\frac{e^{s(h_i, v_k)}}{\sum_j^P e^{s(h_i, v_j)}} \right], \tag{2.11}$$

where for a sample x_i with embedding h_i in class k, v_k is the prototype of class k in the latent space, P is the number of known classes, B is the batch size, and the similarity score

$$s(h_i, v_j) = \frac{h_i^\top v_j}{\|h_i\|\|v_j\|} \cdot \frac{1}{\tau} \tag{2.12}$$

is a temperature-scaled cosine similarity with temperature τ. Class prototypes v_k are computed as the mean embedding of samples in the batch belonging to class k. The combined objective $\mathcal{L} = \mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{contrastive}}$ encourages the encoder to produce a latent space that simultaneously preserves enough information to reconstruct the input (via $\mathcal{L}_{\text{MSE}}$) and clusters same-class samples tightly while pushing different-class samples apart (via $\mathcal{L}_{\text{contrastive}}$).

Nearest Class Mean Classifier with Hybrid Distance

Once the embedding network is trained, it is used to fit a Nearest Class Mean (NCM) classifier [4] that decides which known class an incoming sample belongs to or whether it should be flagged as drifted. NCM is a distance-based classifier that assigns each incoming sample to the class with the closest prototype, here in latent space. Its accuracy depends critically on the choice of distance metric. For each known class c, the prototype is computed as the mean of the embeddings of its training samples:

$$\boldsymbol{h}_c = \frac{1}{n_c} \sum_i [y_i = c]\, \mathbf{h}_i, \tag{2.13}$$

where n_c is the number of training samples observed for class c; this $\boldsymbol{h}_c$ is the full-data counterpart of the per-batch prototype v_k in Eq. (2.11), both being the latent-space centroid of a class. The assigned class for an incoming sample with embedding $\mathbf{h}_j$ is the one minimising a hybrid distance,

$$c_j^* = \arg\min_{c \in C} D(\mathbf{h}_j, \boldsymbol{h}_c), \tag{2.14}$$

$$D(\mathbf{h}_j, \boldsymbol{h}_c) = d_R(\mathbf{h}_j, \boldsymbol{h}_c) + \lambda_1(d_E(\mathbf{h}_j, \boldsymbol{h}_c)), \tag{2.15}$$

where d_E is the Euclidean distance between the sample and the class prototype in latent space, d_R is a Riemannian divergence (geometric distance) between them, $\lambda_1 > 0$ is a tuning parameter controlling their relative contribution, and C is the set of known class indices.

The two terms in the metric are intended to capture both instance and group level fidelity. The Euclidean term enforces instance level fidelity, while the Riemannian term enforces group level

fidelity. Their combination is claimed to be robust against adversarially crafted inputs by exploiting three properties of adversarial subspaces [4]: (a) they are found in lower-probability regions compared to the data manifold; (b) their class distributions tend to vary from their closest natural sub-manifold; and (c) an adversarial sample's nearest neighbour is its unperturbed source, rather than any other point in the training or test set.

The same hybrid distance is also used to identify drifted samples. A sample is taken as evidence of a previously unobserved class when its distance to the nearest known prototype is large. Concretely, a sample x_j with embedding $\mathbf{h}_j$ is flagged as drifted when its minimum hybrid distance to the known prototypes – which we refer to as its *drift score* – exceeds a drift threshold T_{drift},

$$\min_{c \in C} D(\mathbf{h}_j, \boldsymbol{h}_c) > T_{drift}, \tag{2.16}$$

where D is the hybrid distance of Eq. (2.15) and C is the set of known class indices. Kuppa and Le-Khac [4] do not formally specify how T_{drift} should be derived. We describe the calibration procedure adopted in this thesis in Chapter 5.

Concept Discovery

Drifted samples alone do not invalidate the model; instead, the detector treats them as candidate evidence of a new class. Drifted embeddings $\{\mathbf{z}^t_{1_{drift}}, \mathbf{z}^t_{2_{drift}}, \dots\}$ produced during a time window t are buffered, and the detector maintains a running accumulator

$$\Delta^{t \to t+1}_{drift} = \boldsymbol{h}^{t+1}_c - \boldsymbol{h}^t_c = \Delta^{t-1 \to t} + \delta^{t-1 \to t}, \tag{2.17}$$

$$\delta^{t-1 \to t}_i = \mathbf{z}^t_{i_{drift}} - \mathbf{z}^{t-1}_{i_{drift}} \; y_i \in c^t. \tag{2.18}$$

Intuitively, $\delta^{t-1 \to t}_i$ is the displacement of each drifted embedding between successive windows, and Δ_{drift} accumulates these displacements over time. Once the resulting magnitude is large enough, the buffered samples are taken to form a coherent new concept. When the accumulated displacement crosses a threshold T,

$$\|\Delta^{t \to t+1}_{drift}\| > T, \tag{2.19}$$

new mean class prototypes $h_{new,i}$ are registered, both the accumulator and buffer are reset, and the autoencoder is retrained on new data that includes the newly identified class. The paper does not specify how past concepts in the dataset are maintained under bounded memory across retraining cycles. A natural choice, following the rehearsal principles in Section 2.4, is to represent each registered class by a fixed-size exemplar set selected via herding (Eq. (2.9)); we adopt this in Chapter 5.

Operational Protocol

The detector therefore both decides when adaptation is required (via the drift threshold T_{drift} and the concept discovery threshold T) and identifies which incoming samples were candidate evidence for that adaptation (via the concept-discovery buffer). What is not specified by Kuppa and Le-Khac [4] is how this adaptation interacts with any downstream classifier, nor how the candidate evidence should be combined with retained known-class data when the encoder is retrained. We return to these architectural questions in Chapters 3 and 5.

2.6 Summary

Three threads from this chapter carry into the rest of the thesis. First, a drift detector is a *control mechanism* that decides when a learner adapts (Section 2.2), and classical detectors weigh sensitivity against specificity. Therefore, an adversary who can shape the monitored signal can force or suppress the detector's decision. Second, the class-incremental learning literature [15, 16] offers rehearsal with herding (Eq. (2.9)) as a way for a learner with bounded memory to avoid catastrophic forgetting, so we adopt this as a design decision in Chapter 5. Third, the target of this work is the Contrastive NCM detector of Kuppa and Le-Khac [4]: a contrastive autoencoder feeding a hybrid-distance NCM classifier that flags drifted samples (Eq. (2.16)) and a concept-discovery accumulator that registers a new class once the buffered displacement crosses a threshold

(Eq. (2.19)). It is reported to be robust when evaluated as a stand-alone classifier. However, it leaves three components unspecified. Namely, how the thresholds T_{drift} and T are calibrated, how past concepts are retained across retraining, and how the detector couples to a downstream learner. These gaps are exactly what Chapters 3 and 5 make concrete, and the coupling they expose is where this thesis locates the attack surface.

Chapter 3

Target System and Threat Model

This chapter formalises the adaptive learning pipeline targeted in this research and defines the adversary attempting to compromise it. We first describe the architecture of the target system, a static base learner coupled to an adversarially robust drift detector, and then establish the threat model under which the attacks of Chapters 4 and 6 are mounted.

3.1 Target System Architecture

The system under evaluation addresses concept drift in the typical way: by augmenting the base learner with a drift detector, as mentioned in Section 2.2. However, this approach differs in that the supervisory module is dynamic. While traditional detectors rely on fixed statistical thresholds or heuristics, this architecture uses a trainable drift detector. It learns a compressed latent representation of the known data distribution and measures deviation using class prototypes in that space to identify distributional shifts. This approach is intended to provide robustness to adversarial manipulation, a property which is scrutinised in the following chapters of this thesis.

The two components of the system, and the manner in which they are integrated, are described below.

3.1.1 The Base Learner

The base learner $\mathcal{M}$ is a binary multi-layer perceptron trained to classify network flows as either Benign (class 0) or Attack (class 1), taking a d-dimensional feature vector as input:

$$\mathcal{M} : \mathbb{R}^d \rightarrow \mathbb{R}^2$$

3.1.2 The Drift Detector

The drift detector $\mathcal{D}$ is the Contrastive NCM detector of Kuppa and Le-Khac [4], whose components are described in Section 2.5.2. Two thresholds govern its behaviour: the drift threshold T_{drift}, used to flag individual samples as drifted; and the concept threshold T, used to register a new class once the accumulated displacement of buffered drifted embeddings crosses it. Neither is specified a priori by [4]; both are calibrated on a held-out sample of known-class data, with the specific procedure detailed in Chapter 5.

3.1.3 System Integration and Adaptation Cycle

The system couples $\mathcal{D}$ and $\mathcal{M}$ through a trigger-dependent adaptation mechanism: in general $\mathcal{M}$ is static, only being updated when $\mathcal{D}$ registers a new concept. Following the concept-discovery mechanism of Section 2.5.2, $\mathcal{D}$ flags drifted samples in each batch and accumulates them in the concept-discovery buffer. In the formulation of Kuppa and Le-Khac [4], a new concept is registered once the accumulated displacement crosses T (Eq. (2.19)). In our coupled system, this displacement test is necessary but not sufficient: a retraining event fires only when the buffer also holds a

minimum *count* and *fraction* of attack-labelled drifted samples. These two floors complete the paper's underspecified firing rule; we state them structurally here and give their values and empirical motivation in Chapter 5. More generally, the coupling operates through two channels: a *trigger channel*, deciding when a retraining event fires, and a *composition channel*, governing what data $\mathcal{D}$ and $\mathcal{M}$ are retrained on. When the event fires, the coupling layer composes that retraining set from the concept-discovery buffer together with a retained set of per-class exemplars which, under an *accumulate, never-evict* policy, are kept across all later rebuilds. As with the firing floors, we state this composition structurally here and motivate it in Chapter 5.

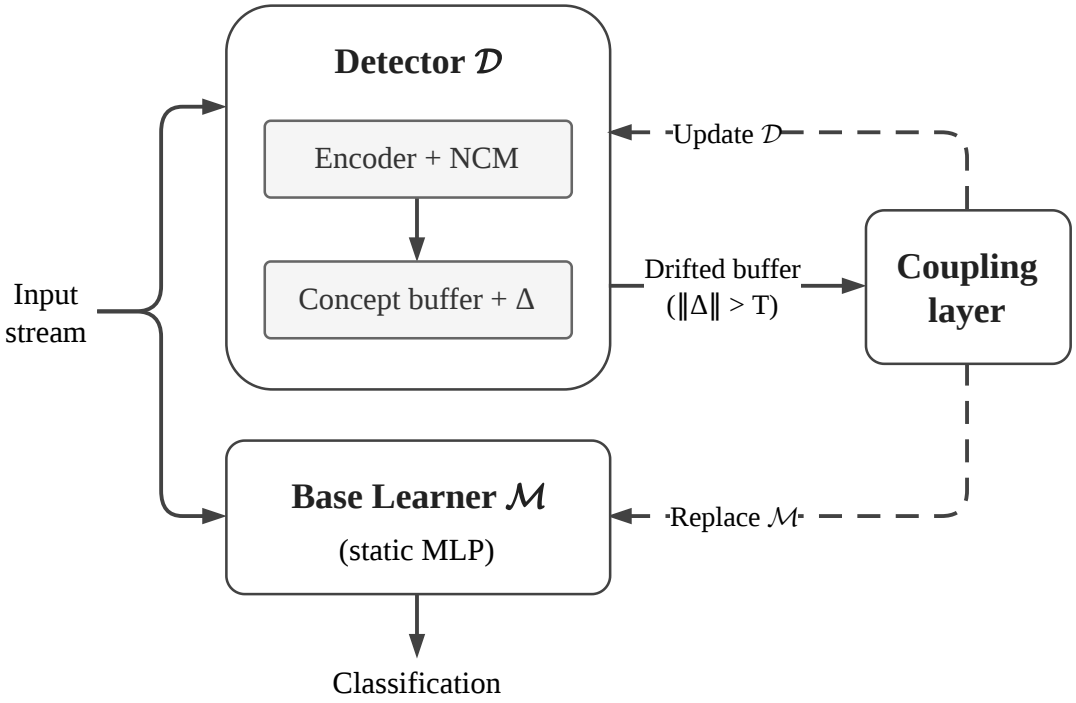

Figure 3.1: The architecture of the target system. $\mathcal{D}$ and $\mathcal{M}$ consume the input stream in parallel: $\mathcal{M}$ produces classifications while $\mathcal{D}$ monitors for drift, accumulating flagged samples in the concept-discovery buffer. When the paper's concept discovery condition is met ($\|\Delta\| > T$), the coupling layer (detailed in Chapter 5) evaluates a set of additional conditions and, only if these are satisfied, triggers a retraining event (dashed): a new prototype is registered, and the encoder and $\mathcal{M}$ are replaced by new models trained on a new dataset composed by the coupling layer.

3.2 Threat Model

To systematically evaluate the security of the target system, we define the parameters under which an adversary operates. We specify their objectives, their knowledge of the system, and their practical capabilities within the environment.

We model the input as a stream of mini-batches $\mathcal{B}_1, \mathcal{B}_2, \ldots$, where each batch $\mathcal{B}_t = \{(x_i, y_i)\}_{i=1}^{n_t}$ contains n_t network flows with feature vectors $x_i \in \mathbb{R}^d$ and labels $y_i \in \{0, 1\}$ (BENIGN or ATTACK). The adversary does not overwrite genuine incoming instances; rather, they inject additional crafted flows into the stream, bounded by a per-batch injection budget.

Definition 3.1 (Injection budget)**.** For an injection rate $p \in [0, 1]$, the adversary may add up to $\lfloor p\, n_t \rfloor$ crafted flows to each batch. Writing $\mathcal{A}_t \subset \mathbb{R}^d \times \{0, 1\}$ for the injected set, the set of admissible poisoned batches obtainable from a clean batch $\mathcal{B}_t$ is

$$\mathcal{T}_p(\mathcal{B}_t) = \{\, \mathcal{B}_t \cup \mathcal{A}_t \;:\; |\mathcal{A}_t| \leq \lfloor p\, n_t \rfloor \,\}. \tag{3.1}$$

Because injected flows augment rather than replace the batch, an injection rate p contaminates a fraction $p/(1+p)$ of the resulting batch. Sustaining an attack across a window of W consecutive

batches costs at most $p \sum_{t=1}^{W} n_t$ injected flows in total; we characterise this budget empirically in Chapter 6.

3.2.1 Attacker Objectives

The primary objective of an adversary is to degrade the predictive performance of the coupled system on genuine traffic. Since $\mathcal{M}$ is static between retraining events, it cannot be corrupted directly. The adversary can act only through the coupling layer of Section 3.1.3 – that is, through its trigger and composition channels. This manifests in two distinct ways:

- *Forcing false adaptation*: the adversary piggybacks on a genuine retraining event or induces the coupling layer to fire while the underlying distribution is stable, causing $\mathcal{M}$ to be rebuilt on adversary-influenced data and consuming computational resources.
- *Impairing adaptation*: the adversary masks genuine drift so that the coupling layer does not fire when it should, leaving $\mathcal{M}$ stale. A subtler variant lets the event fire but biases the composition of the retraining set so that the rebuilt $\mathcal{M}$ is itself degraded.

3.2.2 Attacker Knowledge

The viability of an attack depends heavily on the adversary's understanding of the target system. We frame the attacker's knowledge under three assumptions:

- *White-box*: the adversary possesses complete knowledge of the system, including $\mathcal{D}$'s encoder weights and class prototypes, the current parameters of $\mathcal{M}$, the batch size, the coupling layer's firing conditions, and the calibrated values of the thresholds T_{drift} and T.
- *Grey-box*: the adversary knows the general architecture, that a Contrastive NCM detector $\mathcal{D}$ is coupled to an MLP base learner $\mathcal{M}$ via the adaptation cycle of Section 3.1.3, and the features of the stream, but lacks real-time access to the internal weights, the concept-discovery buffer, or the exact threshold and gate values.
- *Black-box*: the adversary is unaware of the specific detector or base learner implemented. They observe only the output predictions and the raw incoming stream, and must infer system behaviour from changes in its external responses.

3.2.3 Attacker Capabilities

The adversary is assumed to have write access to the data ingestion pipeline, allowing them to inject crafted flows into the stream before it reaches the system, subject to the budget of Definition 3.1. Two manipulation capabilities are available:

- *Label flipping*: assigning an injected flow a label that contradicts its true class.
- *Feature manipulation*: perturbing the feature vector of an injected flow within a bounded budget; exercised to craft known-class flows that cross $\mathcal{D}$'s drift threshold without forming a genuine new class. Constraining such perturbations to protocol-valid flows is left to future work (Chapter 7).

The budget $p < 1$ reflects the assumption that an adversary cannot overwrite the entire stream: doing so would be trivially detectable by upstream anomaly detection.

3.2.4 Adopted Threat Model

The experiments of Chapter 6 operate primarily under the grey-box setting. The adversary knows the coupled $\mathcal{D}$–$\mathcal{M}$ architecture and the structure of the adaptation cycle, but has no access to the live encoder weights, the prototype set, the contents of the concept-discovery buffer, or the calibrated threshold and gate values. Within this setting we exercise both primitives of Section 3.2.3: label-flipped injection (for the poisoning and suppression attacks) and feature-space perturbation (for the false-positive attack on $\mathcal{D}$).

3.3 Summary

This chapter fixed the system and adversary that the rest of the thesis builds upon. The target is a static binary MLP base learner $\mathcal{M}$ coupled to the Contrastive NCM detector $\mathcal{D}$, where $\mathcal{M}$ is rebuilt only when $\mathcal{D}$ registers a concept (Section 3.1.3). Because $\mathcal{M}$ is static between rebuilds, it cannot be corrupted directly: every attack must act through the coupling. The adversary injects rather than overwrites stream flows, bounded by a per-batch budget p (Definition 3.1) that contaminates a fraction $p/(1+p)$ of each batch, and the experiments adopt the *grey-box* model: the architecture and adaptation cycle are known, but the live weights, concept-discovery buffer, and calibrated thresholds are not. Two primitives are available under this model – label flipping and bounded feature perturbation – and Chapter 4 organises the attack space around the two channels they reach.

Chapter 4

Attack Taxonomy and Objectives

This chapter enumerates the attacks available against the coupled system and states precisely which of them we evaluate. As established in Chapter 3, the adversary cannot reach the base learner $\mathcal{M}$ directly ($\mathcal{M}$ is static between retraining events), so every attack must act through the coupling layer. That layer exposes two channels: the trigger channel and the composition channel. We therefore organise the attack space around these two channels rather than around the two components ($\mathcal{D}$ and $\mathcal{M}$) in isolation, because it is the channels, not the components, that the coupling exposes and that the experiments of Chapter 6 target.

We first locate the classical adversarial-learning primitives (evasion, poisoning, and backdooring) within the two channels, then discuss attacks on the trigger channel and the composition channel in turn. We then examine how long an attack persists, distinguishing payloads that the next retraining event overwrites from those the exemplar memory makes durable, before describing the compound attacks that combine both channels. Finally, it states which attacks this thesis evaluates and which are deferred to future work.

4.1 The Attack Surface of the Coupling Layer

Recall from Section 3.1.3 that the coupling between $\mathcal{D}$ and $\mathcal{M}$ operates through two channels.

The Trigger Channel

The trigger channel governs *when* the system rebuilds. A retraining event fires only when the concept-discovery buffer satisfies the firing predicate, a conjunction of three conditions: (i) the accumulated displacement of the buffered drifted embeddings crosses the concept threshold, $\|\Delta\| > T$ (Eq. (2.19)); (ii) the buffer holds at least a minimum number of attack-labelled drifted samples; and (iii) those samples make up at least some fraction of the buffer. The first condition is paper-faithful; the count and fraction conditions are a refinement we established in Section 3.1.3 and detailed in Chapter 5. The trigger channel thus presents three distinct sub-surfaces: *displacement*, *count*, and *fraction*. By targeting one or more of these, an adversary can force a retraining event that genuine conditions do not warrant, or suppress one that they do.

The Composition Channel

The composition channel governs *what data* the rebuilt models are trained on. When a retraining event fires, the encoder and $\mathcal{M}$ are rebuilt on a dataset that the coupling layer composes from the concept-discovery buffer (the drifted samples $\mathcal{D}$ has flagged) together with a retained set of per-class exemplars. An adversary who places crafted flows in the buffer therefore controls part of the training set on which the system rebuilds.

These two channels are exhaustive for the threat model of Section 3.2.3. That is, an adversary restricted to injecting stream data can influence the coupled system only by changing whether a retraining event fires (the trigger channel) or which data that event consumes (the composition channel). The classical adversarial-learning primitives are not discarded by this view; each can

be seen as an action on one of the two channels, as summarised in Table 4.1. We use *evasion* to refer to attacks that change a decision without altering model weights, *poisoning* for attacks that corrupt a model through its training data, and *backdooring* for targeted poisoning that implants a hidden, trigger-activated behaviour. The trigger channel is therefore attacked by *evasion* alone, while *poisoning* and *backdooring*, whether aimed at $\mathcal{D}$ or $\mathcal{M}$, act through the composition channel, since model weights change only at a rebuild. Because $\mathcal{D}$ and $\mathcal{M}$ are rebuilt from the same buffer, a single composition attack can corrupt both at once.

Primitive	Channel	Effect on the coupled system
Evade $\mathcal{D}$	Trigger	Force (false positive) or suppress (false negative) a retraining event
Poison $\mathcal{D}$	Composition	Corrupt the encoder or prototypes so the firing decision is systematically wrong
Backdoor $\mathcal{D}$	Composition	Implant on-demand control of the firing decision
Poison $\mathcal{M}$	Composition	Bias the retraining set so the rebuilt $\mathcal{M}$ misclassifies
Backdoor $\mathcal{M}$	Composition	Implant a feature-triggered backdoor through the retraining set

Table 4.1: The classical adversarial-learning primitives, recast as actions on the two channels exposed by the coupling layer. Organising by channel (rather than by target component) foregrounds the attack surface that the coupling actually exposes.

4.2 Attacks on the Trigger Channel

The trigger channel decides *when* the system rebuilds. An adversary targeting it pursues one of two opposite goals: forcing a false retraining event under stable conditions, or suppressing one that should occur under genuine drift.

4.2.1 Forcing Adaptation (False Positives)

Here the adversary forces the firing predicate to evaluate to true while the underlying distribution is stable. A forced event can be harmful for two reasons. First, retraining is expensive, so a sustained stream of forced retraining events is a denial-of-service vector. Second, and more consequentially, because the adversary chooses the moment of retraining, they can take control of the composition channel concurrently, to supply adversarial data which the forced rebuild trains on (the compound attack of Section 4.5). A false positive can be induced through any of the firing sub-surfaces:

- *Displacement, via feature perturbation*: an adversary with the feature-manipulation capability of Section 3.2.3 can perturb known-class flows so that their embeddings cross the drift threshold T_{drift} (Eq. (2.16)), effectively manufacturing drifted samples from an unchanged distribution. However, forcing an actual concept-discovery event additionally requires displacing the drifted-batch mean past T. This means the perturbed samples must appear somewhat coherent rather than just individually drifted. We realise this attack and assess its feasibility in Chapter 6.
- *Count and fraction floors*: injecting genuinely novel flows under their true (attack) label during an otherwise-stable window satisfies the count and fraction conditions and forces an early, attacker-timed retraining event.

4.2.2 Suppressing Adaptation (False Negatives)

Here the adversary prevents the predicate from firing when genuine drift occurs, leaving $\mathcal{M}$ stale and unable to classify the emerging concept correctly. Again either sub-surface can be exploited:

- *Fraction floor*: injecting benign-labelled flows alongside genuine drift keeps the buffer's attack fraction below the threshold; the displacement may accumulate, but the gate never fires.
- *Displacement*: shaping the buffer so that $\|\Delta\|$ never crosses T during a genuine shift.

Persistent suppression means keeping this false negative going for as long as the new class keeps arriving. If the predicate never fires, $\mathcal{M}$ is never rebuilt, so it never learns the new class and keeps misclassifying it. Because $\mathcal{D}$ and $\mathcal{M}$ change only at a retraining event, the adversary does not have to overpower a model that corrects itself as the stream flows past; they need only keep the single trigger from firing. The mechanisms above are all evasion, which is inherently transient, meaning the moment the injection stops, the next genuine drift fires a retrain and the staleness is gone. A durable suppression, one that corrupts $\mathcal{D}$ so that it stops flagging the drift at all, is not an evasion but a composition-channel attack on the detector. We discuss it in Section 4.3.3, and follow up on the transient/durable distinction in general in Section 4.4.

4.3 Attacks on the Composition Channel

The composition channel decides *what* data the models are rebuilt on. Because the rebuild trains both the encoder ($\mathcal{D}$) and $\mathcal{M}$ on that data, a composition attack can corrupt either component, or both at once. Attacks here can only occur in conjunction with a retraining event, whether genuine or forced, because the buffer is consumed only when the trigger fires. This is precisely why the two channels are so often combined (Section 4.5).

4.3.1 Poisoning $\mathcal{M}$

When a retraining event fires, $\mathcal{M}$ is rebuilt on the concept-discovery buffer together with the retained exemplars. An adversary who has placed mislabelled flows (genuine ATTACK flows presented as BENIGN) in the buffer forces $\mathcal{M}$ to learn their features as BENIGN, raising its false-negative rate after the rebuild. Because the encoder retrains on the same buffer, this payload can simultaneously degrade $\mathcal{D}$, a representation-level effect we examine alongside $\mathcal{M}$'s degradation in Chapter 6.

The attack as we evaluate it is dirty-label: it depends on presenting attack traffic under the wrong label. A clean-label counterpart is equally admissible under our threat model, which already grants feature manipulation (Section 3.2.3): the adversary perturbs the features of correctly labelled flows so that they misrepresent the true class and the rebuilt $\mathcal{M}$ misclassifies them. Clean-label poisoning is stealthier, since the labels stay consistent, but generally harder to land; we do not evaluate it and record it as future work (Chapter 7).

4.3.2 Backdooring $\mathcal{M}$

A targeted variant of poisoning injects a feature trigger paired with a target label, so that the rebuilt $\mathcal{M}$ behaves normally on clean inputs but as the adversary specifies on any inputs carrying the trigger. We evaluate this type of attack against $\mathcal{M}$ in Section 6.4.

4.3.3 Poisoning and Backdooring $\mathcal{D}$

The same poisoned buffer that impacts $\mathcal{M}$ also retrains the encoder and prototypes, so the composition channel can be used to attack the detector itself. Where a poisoned $\mathcal{M}$ misclassifies, a poisoned $\mathcal{D}$ corrupts the firing decision. If tuned correctly, $\mathcal{D}$ ceases to flag a chosen drift, giving a durable form of the suppression of Section 4.2.2. A backdoored $\mathcal{D}$ contains a feature trigger that toggles the firing decision, letting the adversary flag or hide drift at will. Whether the effects of either persist for free is governed by Section 4.4. We evaluate neither, and record them as future work (Chapter 7).

4.4 Attack Persistence and Maintenance

Attacks differ not only in which channel they use but in how long their effect lasts after the adversary's actions. Because a rebuild retrains $\mathcal{D}$ and $\mathcal{M}$ from the concept-discovery buffer together with the accumulated, never-evicted exemplar set (Section 3.1.3), an attack's persistence depends on which of these its payload reaches. We distinguish two cases.

A *transient* payload lives only in the buffer or in a single rebuild. The buffer is reset after every event and $\mathcal{M}$ is retrained from its initial state, so a transient payload is overwritten by the next genuine retraining event. The evasion-style trigger-channel attacks of Section 4.2 are transient in

this sense. A suppression lapses the moment the adversary stops injecting, and a forced event corrupts only the rebuild it triggers. To keep such an attack effective the adversary must *maintain* it, continuously suppressing genuine events or re-applying the payload at each event they force.

A *durable* payload, by contrast, is one the system writes into its long-term memory: a registered adversarial prototype or the per-class exemplars harvested at concept discovery. Because the never-evict policy replays these at every subsequent rebuild and never discards them, such a payload persists with no further action from the adversary. The rehearsal mechanism that protects the system against catastrophic forgetting (Section 2.4) thereby protects an adversarial concept just as faithfully as a genuine one. It essentially prevents the system from *healing*.

The cost of an attack is correspondingly bimodal. Transient effects demand a *sustained* injection rate, maintained for as long as the effect is to last; durable effects demand only a *one-off* cost, reaching the long-term memory once. In Chapter 6 we characterise the sustained budget of the suppression attack and test which payloads, once harvested into the exemplar memory, survive subsequent clean retraining events.

4.5 Compound Attacks

A compound attack combines an action on the trigger channel (controlling *when* a rebuild occurs) with one on the composition channel (controlling *what* is learned). The primary compound attack is Evade $\mathcal{D}$ + Poison $\mathcal{M}$: the adversary forcefully triggers a retraining event and supplies the poisoned buffer concurrently. This grants them control over the exact timing of the attack, but requires a working false-positive mechanism, whose feasibility Chapter 6 examines and finds costly.

However, a compound attack is often unnecessary; the adversary can achieve the same poisoning by injecting the payload only when the system would have retrained anyway. This is simply a composition-channel attack of Section 4.3.1 executed opportunistically, cheaper and stealthier than a forced compound and requiring no false-positive capability.

The remaining cells of the combined space are either analogues of the above (e.g. Evade $\mathcal{D}$ + Backdoor $\mathcal{M}$, which substitutes a backdoor for general degradation) or are deemed to be impractical or pointless against this architecture. In particular, simultaneously backdooring both $\mathcal{D}$ and $\mathcal{M}$ through a single stream is of questionable feasibility, and pairing an inference-time evasion of $\mathcal{M}$ with an attack on $\mathcal{D}$ adds no value unless the goal is simply a mass-evasion campaign. We do not pursue these.

4.6 Summary

This chapter organised the attack space around the coupling rather than the system's components. Because $\mathcal{M}$ is static between rebuilds, every attack acts through one of two channels: the *trigger* channel, attacked by evasion to force or suppress a retraining event (Section 4.2), and the *composition* channel, attacked by poisoning or backdooring whatever a rebuild trains on (Section 4.3). An attack's effect is further either *transient* (requiring sustained maintenance) or *durable* (lodged in the accumulated, never-evicted exemplar memory) (Section 4.4).

This taxonomy spans the full coupling-layer attack space. This thesis evaluates a focused subset of it, chosen to characterise the two channels rather than to demonstrate every cell. Table 4.2 maps each evaluated attack to the channel and sub-surface it tests and to the research question under which it is examined. The remaining attacks are identified as future work. Throughout, we hold to the framing established in Chapter 3: these are attacks on the coupling, and where $\mathcal{D}$ itself is targeted (the feature-perturbation false positive) we say so explicitly. Chapter 5 realises these attacks concretely.

Attack	Channel (sub-surface)	Evaluated in
Suppression	Trigger (fraction floor)	Chapter 6, RQ2–RQ3
Manufactured novelty	Trigger (count/fraction floors)	Chapter 6, RQ3
PGD false positive on $\mathcal{D}$	Trigger (displacement, via features)	Chapter 6, RQ3
Poison $\mathcal{M}$ (riding genuine drift)	Composition	Chapter 6, RQ2
Backdoor $\mathcal{M}$ (feature trigger)	Composition	Chapter 6, RQ2
Poison $\mathcal{D}$ (durable suppression)	Composition	Future work
Backdoor $\mathcal{D}$ (feature trigger)	Composition	Future work
Inference-time evasion of $\mathcal{M}$	Outside the coupling	Future work

Table 4.2: The attacks evaluated in this thesis, located within the channel taxonomy. The selection characterises both coupling channels to some extent. Also identifies attacks left as future work (Chapter 7).

Chapter 5

Implementation

This chapter describes how the target system of Chapter 3, our evaluation pipeline and our attacks are realised in code. Because Kuppa and Le-Khac [4] release neither code nor a precise specification of several components, we first reimplement their Contrastive NCM detector and confirm that the reimplementation reproduces their reported behaviour. We then build the coupled adaptive system in which the detector supervises a downstream base learner, and describe the design decisions that this required. Four of these decisions complete components that the paper leaves underspecified and each was surfaced by a concrete failure of a naive implementation rather than chosen a priori. We frame them as filling gaps in the paper's intended architecture, not as corrections to it. The chapter closes with the implementation of the attacks selected for evaluation in Chapter 4.

5.1 Datasets and Preprocessing

We evaluate on CICIDS2017, a widely used intrusion-detection benchmark of labelled bidirectional network flows captured over five working days [17]. We use the Engelen-corrected release of CICIDS2017 [18], which repairs documented errors in the original dataset's labelling and feature-extraction pipeline. Kuppa and Le-Khac [4] report using this same correction, although their testbed description matches the larger CSE-CIC-IDS2018 dataset[1]. Therefore we read their evaluation as using the same corrected CICIDS2017. However, the comparison in Section 5.2.2 is still indicative rather than exact because the reference implementation, the test-set composition, and the feature subset all remain unspecified.

The five daily captures are concatenated in their natural order. We discard flow-identifier columns (the flow identifier, source and destination addresses and ports, and the timestamp) so that the detector cannot key on artefacts of the capture rather than on actual traffic behaviour. We also consolidate the fine-grained attack labels into their parent families (for example, the several denial-of-service variants into a single `DoS` class). Non-finite feature values are replaced, all-empty columns are dropped, the remaining gaps are mean-imputed, and the features are standardised. We are left with a 77-dimensional input space over seven traffic classes. Kuppa and Le-Khac [4] report a 72-dimensional feature set but do not specify which columns they retain, so we could not match their exact feature space; this is a concrete instance of the unpublished preprocessing noted above.

To simulate concept drift we withhold a subset of classes as *novel*: these never appear in the initial training data and first arrive partway through the stream, while the remaining *known* classes train the initial detector and base learner. Our main experiments withhold `DoS` and `PortScan`; a robustness variant withholds `DoS` and `DDoS` (Chapter 6). The stream preserves the natural Monday-to-Friday ordering rather than being shuffled, so novel traffic arrives at its true point in time mirroring a real-world scenario.

[1] A corrected CSE-CIC-IDS2018 also exists [19], but post-dates [4] and is not the correction they cite.

5.2 Reimplementing and Reproducing the Contrastive NCM Detector

5.2.1 Reimplementation

Because no reference implementation is published, we reimplement the detector of Section 2.5.2 in PyTorch directly from the paper's specification [4]. The embedding network is an autoencoder trained jointly on a mean-squared reconstruction loss and the supervised contrastive loss, with temperature $\tau = 0.1$; it maps the 77-dimensional input to a 32-dimensional latent space through a single 64-unit hidden layer. On these embeddings we fit the Nearest Class Mean classifier with the hybrid distance of Eq. (2.15) ($\lambda_1 = 0.1$), taking each class prototype to be the mean latent vector of its training samples. Concept discovery accumulates the telescoping displacement of successive drifted-batch means as in Eq. (2.19). Following the paper we train the encoder network with the Adam optimiser at a learning rate of 10^{-4} for 300 epochs, on its 25/75 train–test split.

Three components are deliberately left open at this stage because the paper does not specify them: the calibration of the two thresholds (Section 5.4.2), the policy for retaining past concepts (Section 5.4.3), and the exact coupling to a downstream classifier (Section 5.3). We fix the reimplementation against the paper before introducing any of them.

5.2.2 Reproduction

We validate the reimplementation on the paper's drift-detection protocol: a subset of classes is withheld from training, their test-set samples form the drifted (positive) class and the other classes the negatives. Matching the paper's "two classes dropped", chosen at random over five runs, we compare precision, recall and F_1 directly against its reported values (Table 5.1). The plain-autoencoder baseline reproduces almost exactly ($F_1 = 0.788$ against 0.777) and the contrastive detector lands lower (0.887 against 0.942); a threshold-free check agrees, our AUROC of 85.1 tracks the same shortfall against the paper's 89.8. The gap is most plausibly caused by the details the paper leaves unspecified (discussed in Section 5.1), not by the detector itself. The reproduction preserves the paper's ordering, the proposed method ahead of the autoencoder baseline.

Method	Source	Precision	Recall	F_1
Contrastive NCM (proposed)	Ours	0.815	0.979	0.887
	Kuppa and Le-Khac [4]	0.965	0.921	0.942
Plain AE + NCM	Ours	0.666	0.979	0.788
	Kuppa and Le-Khac [4]	0.788	0.766	0.777

Table 5.1: Reproduction of the held-out-class drift-detection benchmark at the paper's "two classes dropped" setting: our precision, recall and F_1 against the values reported by Kuppa and Le-Khac [4] (ours averaged over five random two-class withholdings). The plain-autoencoder baseline reproduces almost exactly on F_1; the contrastive detector lands modestly lower, and both of our detectors trade precision for recall relative to the paper.

Table 5.1 above establishes that the reimplementation detects drift as the paper's detector does; it says nothing about its adversarial robustness, which is the property for which the contrastive detector was originally proposed. We therefore also run the paper's instance-based poisoning experiment, where a growing fraction of adversarially perturbed novel samples is injected into training, and the contrastive detector's accuracy degrades gracefully with the injection ratio, reproducing the trend the paper reports. We stress that this is only a qualitative check. The paper's headline claim is comparative – that the Contrastive NCM detector is more robust than its baselines – and we do not cleanly reproduce that, because the plain-autoencoder baseline sits at the balanced-accuracy floor throughout. We also do not reproduce the paper's concept-based poisoning experiment as it is built on the MAA algorithm [20], a separate black-box manifold-approximation attack whose faithful reimplementation is out of scope.

5.3 The Coupled Adaptive System

5.3.1 Architecture and Streaming Harness

The coupled system realises the architecture of Chapter 3: the detector $\mathcal{D}$ and base learner $\mathcal{M}$ consume the same stream in parallel. $\mathcal{M}$ classifies each batch while $\mathcal{D}$ monitors it for drift, flagging individual samples (Eq. (2.16)) and accumulating them in the concept-discovery buffer. When the firing predicate is satisfied (Section 5.4.1) the coupling layer assembles a retraining set (Section 5.4.3) and rebuilds the encoder, the class prototypes, and $\mathcal{M}$. Algorithm 1 sets out the full per-batch cycle, tying together the four refinements of Section 5.4. A streaming harness drives this loop one batch at a time and records, per batch, the base learner's F_1 and per-class false-negative rates, the number of detector and base-learner retraining events, the detector's drift rate on a fixed novel pool, and every firing decision.

Algorithm 1: Coupled adaptation cycle for one stream batch (AdaptiveDAdaptiveM).

Input: batch (X_t, y_t); detector $\mathcal{D}$ with encoder h and prototypes $\{\boldsymbol{h}_c\}$; base learner $\mathcal{M}$; buffer $\mathcal{B}_{\text{drift}}$ and accumulator Δ; thresholds T_{drift}, T; floors $N_{\min}$, $\varphi_{\min}$.
Output: predictions $\hat{y}$ and per-batch metrics.

$\hat{y} \leftarrow \mathcal{M}(X_t)$
$F \leftarrow \{\, x \in X_t : \min_c D(h(x), \boldsymbol{h}_c) > T_{drift} \,\}$ // Eq. (2.16)
append F with its labels to $\mathcal{B}_{\text{drift}}$, and update Δ_{drift} // Eq. (2.17)

$n_a \leftarrow$ number of attack-labelled in $\mathcal{B}_{\text{drift}}$; $\quad m \leftarrow |\mathcal{B}_{\text{drift}}|$
FIRE $\leftarrow (\|\Delta_{drift}\| > T) \wedge (n_a \geq N_{\min}) \wedge (n_a/m \geq \varphi_{\min})$ // Eq. (5.1)

if FIRE **then**
- register new prototype $\boldsymbol{h}_{\text{new}}$
- $\mathcal{D}_{\text{retrain}} \leftarrow \mathcal{B}_{\text{drift}} \cup \mathcal{E}_{\text{known}} \cup \mathcal{E}_{\text{novel}}$ // prior memory; Eq. (5.4)
- $\mathcal{E}_{\text{novel}} \leftarrow \mathcal{E}_{\text{novel}} \cup \text{herd}(\mathcal{B}_{\text{drift}}, E)$ // fold in new concept; Eq. (5.5)
- rebuild $\mathcal{M}$ and encoder h from scratch on $\mathcal{D}_{\text{retrain}}$
- recalibrate T on the new encoder // Eq. (5.3)
- reset $\mathcal{B}_{\text{drift}} \leftarrow \varnothing$ and $\Delta_{drift} \leftarrow \mathbf{0}$

The harness also exposes the injection hook through which the adversary introduces crafted traffic (Section 5.5). The base learner $\mathcal{M}$ that the detector supervises is an MLP with three hidden layers (256–128–64 units, ReLU) over the same 77-dimensional input, with a binary Benign/Attack output. It is trained under cross-entropy with the Adam optimiser, its initial train running for 100 epochs at learning rate 10^{-3} with inverse-frequency class weights to offset the benign-dominated traffic mixture.

5.3.2 Three System Configurations

We use three configurations of the coupling, which together form an ablation ladder used throughout Chapter 6:

- StaticDStaticM: neither component adapts. $\mathcal{M}$ remains at its initial state and never sees the novel classes. This is the no-adaptation lower bound.
- StaticDAdaptiveM: the detector is frozen but $\mathcal{M}$ adapts on each firing event, isolating the composition channel from any drift in the representation itself.
- AdaptiveDAdaptiveM: both components adapt on each event. This is the paper-faithful system and the default victim.

Fixing which part of the coupling is allowed to adapt lets us attribute observed effects to a specific mechanism rather than to the system as a whole.

5.4 Completing the underspecified Components

Deploying the detector in a coupled setting forced four design decisions that the paper leaves open. We present each as the naive choice, the concrete failure it produced in our implementation, and the refinement that resolves it. We stress that these complete the paper's intended architecture rather than correct an error in it.

5.4.1 The Count-and-Fraction Firing Gate

The paper triggers concept discovery based on displacement alone; that is, a new concept is registered once the accumulated displacement crosses T. Coupled to a retraining loop, this proved far too eager. In an unattacked run, the system's first concept-discovery event fired on a buffer holding only three attack-labelled drifted samples, rebuilding $\mathcal{M}$ on what could effectively be noise. In other words, a handful of borderline samples can satisfy it long before genuine drift is present.

We therefore gate firing on the *makeup* of the buffer in addition to its displacement. Writing n_a for the number of attack-labelled drifted samples in the buffer and m for the total buffer size, a retraining event fires at window t only when

$$\underbrace{\|\Delta^t_{drift}\| > T}_{\substack{\text{displacement}\\ \text{(paper-faithful)}}} \wedge \underbrace{n_a \geq N_{\min}}_{\text{count floor}} \wedge \underbrace{n_a/m \geq \varphi_{\min}}_{\text{fraction floor}}, \tag{5.1}$$

where we use $N_{\min} = 500$ and $\varphi_{\min} = 0.3$; setting both floors to zero recovers the paper's displacement-only rule. Each floor is chosen by its purpose rather than tuned: the count floor is a minimum sample for forming a stable prototype and retraining without fitting to a handful of borderline flows, and the fraction floor is set above the detector's own false-positive contamination of the buffer. Since T_{drift} is calibrated to a roughly 10% known-class false-positive rate (Section 5.4.2), the buffer carries that much benign noise, and a fraction floor comfortably above it ensures a genuine candidate concept dominates the buffer rather than the noise. The precise values are a calibration we do not claim is optimal; Section 7.2 discusses how far the findings depend on them. These two floors are the count and fraction sub-surfaces of the trigger channel named in Section 3.1.3; the trigger-channel attacks of Chapter 6 act on them, and Section 6.5.3 decomposes their distinct roles.

5.4.2 Threshold Calibration

The paper sets the concept threshold $T = 3.5$ a priori, but this value is tied to the scale of its embeddings and does not transfer to ours. We calibrate both thresholds on a held-out sample $\mathcal{X}_{\text{cal}}$ of known-class data. Writing $Q_q(\cdot)$ for the q-quantile of a set and $s(x) = \min_{c \in C} D(h(x), \boldsymbol{h}_c)$ for a sample's drift score (Eq. (2.16)), the drift threshold is the 0.90 quantile of known-class drift scores,

$$T_{drift} = Q_{0.90}\big(\{\, s(x) : x \in \mathcal{X}_{\text{cal}} \,\}\big) = 1.34, \tag{5.2}$$

which fixes the known-class false-positive rate at roughly 10%. The concept threshold is the 0.995 quantile of accumulated drifted-batch displacement norms (Eq. (2.19)), measured over $R = 200$ seeded runs, each accumulating $\Delta^{(r)}$ across $W = 20$ random sub-batches of 256 known-class samples,

$$T = Q_{0.995}\big(\{\, \|\Delta^{(r)}\| : r = 1, \ldots, R \,\}\big) = 37.84. \tag{5.3}$$

The values above are those of the main victim (with novel classes `DoS` and `PortScan` withheld); independently trained detectors calibrate comparable but distinct thresholds under the same rule (the T_{drift} values of the Chapter 6 variants span 1.19–1.34). Within a run, T_{drift} is held fixed, while T is recalibrated on the new encoder at each retraining event.

5.4.3 Retrain-Set Composition and Encoder Stability

When a retraining event fires the encoder must be rebuilt, and the paper does not specify on what data. The naive choice is retraining on the drifted buffer alone; this destabilises the representation. The encoder specifically reorganises its latent space around the newly discovered concept

and known-class performance collapses. This is equivalent to catastrophic forgetting [14], where learning a new task erases earlier ones.

Following the rehearsal principle that mitigates forgetting [15], the coupling layer composes the retraining set from the drifted buffer together with a fixed-size set of per-class exemplars selected via herding (Eq. (2.9)) [16]. We adopt an *accumulate, never evict* policy, meaning once a concept is registered, its exemplars are retained for every subsequent rebuild. Concretely, at the r-th firing the encoder and $\mathcal{M}$ are rebuilt on

$$\mathcal{D}^{(r)}_{\text{retrain}} = \mathcal{B}_{\text{drift}} \cup \mathcal{E}_{\text{known}} \cup \mathcal{E}^{(r-1)}_{\text{novel}}, \tag{5.4}$$

where $\mathcal{B}_{\text{drift}}$ is the concept-discovery buffer holding the just-discovered concept, $\mathcal{E}_{\text{known}}$ the fixed per-class known exemplars, and $\mathcal{E}^{(r-1)}_{\text{novel}}$ the memory of *previously* discovered concepts. The new concept is then folded into that memory for every later rebuild, under the never-evict recursion

$$\mathcal{E}^{(r)}_{\text{novel}} = \mathcal{E}^{(r-1)}_{\text{novel}} \cup \text{herd}\big(\mathcal{B}_{\text{drift}}, E\big), \qquad \mathcal{E}^{(0)}_{\text{novel}} = \varnothing, \tag{5.5}$$

with $\text{herd}(\,\cdot\,, E)$ selecting the $E = 500$ samples closest to the newly registered prototype (Eq. (2.9)). Drawing on $\mathcal{E}^{(r-1)}_{\text{novel}}$ rather than $\mathcal{E}^{(r)}_{\text{novel}}$ avoids double-counting the fresh concept, which already enters through $\mathcal{B}_{\text{drift}}$. The recursion is monotone (nothing is ever removed) which is exactly what makes a registered concept, adversarial or genuine, durable. This realises the composition channel concretely, and it has a direct consequence for adversarial persistence (Chapter 4). The same rehearsal that protects a genuine concept against forgetting protects an adversarial one just as faithfully, preventing the system from "healing" on its own once a payload has been registered.

In our implementation both components are rebuilt *from scratch* on a firing event, re-initialised and re-trained under the same protocol used at initialisation, following the paper [4]. The base learner $\mathcal{M}$ is re-initialised and trained for 100 epochs, with inverse-frequency class weights, on the binary set formed from the drifted buffer, the per-class known exemplars, and the accumulated novel exemplars. The labels used are the ones observed, including any the adversary has forged. The encoder is likewise re-initialised and trained for 300 epochs under the same contrastive loss on the same data with its multiclass labels, with the drifted buffer capped at 5000 samples and assigned the newly registered prototype and the known exemplars repeated twice to rebalance the loss against the buffer. Each discovered concept contributes up to 500 herding-selected exemplars to the accumulated novel set, which under the never-evict policy is retained at every subsequent rebuild.

5.4.4 Evaluation Surface

Finally, the coupling forces a choice of what to measure $\mathcal{M}$ on. We evaluate it on a fixed, class-balanced *benchmark* set that is aware of the novel classes, sampled at the end of each day, which captures the learner's standing accuracy as it adapts over the week. Crucially, we report per-class false-negative rates alongside aggregate F_1, because, as the attacks of Chapter 6 show, aggregate F_1 can remain almost unchanged while the false-negative rate on a single attack class rises sharply.

With these four refinements in place, the coupled system adapts correctly under genuine drift: on an unattacked run it discovers `DoS` (though not `PortScan`, Section 6.4), retrains once, and reaches a Friday benchmark F_1 of 0.71 against a no-adaptation floor of 0.207. This working configuration is the baseline against which Chapter 6 measures every attack.

5.5 Attack Implementation

We close with the implementation of the attacks selected for evaluation in Chapter 4; their results are reported in Chapter 6. We organise them by adversarial objective rather than by channel, because a single mechanism can act through more than one channel, and the objective is what the evaluation actually measures. All four attacks are instances of the shared injection model described next.

5.5.1 The Injection Model

The adversary acts by injecting crafted flows into the stream. We model this as a per-batch *injection* rather than replacement: at injection fraction p the attacker adds $\lfloor p\, n_t \rfloor$ samples to a batch of n_t genuine samples, so the crafted flows make up a fraction $p/(1+p)$ of the batch (Definition 3.1). The injected flows are drawn with replacement from the withheld novel pool, then concatenated into the batch and shuffled; injection is also gated to begin only after the all-benign Monday warm-up. Two further degrees of freedom complete the model. First, the attacker chooses the *label* under which the injected flows are presented, which determines both whether they push the firing gate towards or away from a fire and what $\mathcal{M}$ learns if a rebuild does occur. Second, the injection rate may be varied over time through a *schedule* function, supporting sustained and time-varying injection. Each attack below is an instance of this model under a particular label and schedule; the feature-space attack of Section 5.5.3 additionally perturbs the injected flows themselves.

5.5.2 Suppressing Adaptation

The simplest way to deny adaptation is to hold the firing gate shut. The gate's two floors on the drifted samples mean it fires only once at least 500 of them have accumulated and they make up at least 30% of the buffer (Section 5.4.1). The per-instance drift threshold, however, is label-independent, meaning a novel sample is flagged as drifted whether the adversary presents it as benign or as attack. Injecting novel flows under a benign label therefore admits them to the buffer, inflating its size, while withholding them from the attack-labelled count, which drives the attack fraction below the 0.3 floor. If this is sustained at a sufficient rate the gate can be held shut for the whole stream, so no concept-discovery event fires, $\mathcal{M}$ is never rebuilt, and the emerging class is never learned. The attack crafts no features, relying only on the ability to inject benign-labelled flows.

5.5.3 Forcing Adaptation

Forcing is the dual objective: rather than starving the gate, the adversary drives it past its thresholds to trigger an attacker-timed rebuild – either earlier than genuine drift would, or on an otherwise stable distribution. Because the gate is a conjunction (Section 5.4.1), a forcing attack must satisfy all three conditions at once, and this rules out any displacement-only manoeuvres. Injecting drifted flows under a benign label does move the accumulated displacement but, carrying no attack label, such flows fail the count and fraction floors. Forcing therefore requires perturbing flows that are injected under the attack label, so that they raise the floors while also displacing the batch mean. We realise this with a feature-space attack.

The feature-space attacks perturb the injected flows directly and come in three variants of increasing sophistication. A *per-sample* variant maximises each flow's individual drift score. It saturates instance level drift detection, but because each embedding is pushed independently the batch mean barely moves, so it does *not* force a fire – this is the basis of the "fooling detection is not forcing" result of Section 6.5.1. A *directional* variant instead pushes every embedding along a single fixed (but random) latent direction, producing a coherent batch-mean shift. This can cross T, but only at a large budget, and serves as the naive baseline. The *mean-shift* variant optimises that direction rather than fixing it, and is the efficient forcing attack.

All three variants use projected gradient descent on a fixed attack pool of 2000 held-out known-class flows. Each flow is updated for $K = 50$ steps under an L_∞ budget ϵ (swept in Chapter 6),

$$x^{(k+1)} = \mathrm{clip}\Big(\Pi_{\|\cdot - x^{(0)}\|_\infty \le \epsilon}\big(x^{(k)} + \alpha\, \mathrm{sign}(\nabla_x \mathcal{L}(x^{(k)}))\big)\Big), \qquad \alpha = \frac{2.5\,\epsilon}{K}, \tag{5.6}$$

where Π projects onto the ϵ-ball around the clean flow $x^{(0)}$ and clip restricts each feature to its observed range so the crafted flows stay plausible. The variants differ only in the objective $\mathcal{L}$ they ascend. The *per-sample* variant maximises each flow's own drift score, $\mathcal{L}_{\mathrm{ps}}(x) = \min_{c \in C} D(h(x), \boldsymbol{h}_c)$; this saturates instance-level detection, but because each embedding moves independently it leaves the displacement accumulator almost unchanged. The *directional* variant pushes every embedding along one fixed random unit direction u, $\mathcal{L}_{\mathrm{dir}}(x) = \langle h(x), u \rangle$, producing a coherent shift that can cross T but only at a large budget. The *mean-shift* variant splits the pool into two halves A and B

and jointly optimises their perturbations to maximise the separation of the encoded batch means,

$$\mathcal{L}_{\text{ms}} = \left\|\mu_A - \mu_B\right\|, \qquad \mu_A = \tfrac{1}{|A|}\sum_{x\in A} h(x), \quad \mu_B = \tfrac{1}{|B|}\sum_{x\in B} h(x), \tag{5.7}$$

which is exactly the displacement the concept-discovery accumulator integrates. The perturbed flows are injected under the attack label so that they also satisfy the gate's count and fraction floors. Each variant is run white-box, perturbing against the victim detector itself, and grey-box, perturbing against a surrogate detector and transferring the crafted flows to the victim. The surrogate shares the victim's architecture, hyperparameters and withheld classes but is trained on a disjoint partition of the known-class data, sharing no training flows with the victim (Section 6.5.1). Therefore it models an adversary who knows the system design and the data distribution and trains their own detector, rather than one with access to the victim's data.

5.5.4 Poisoning the Base Learner

Where suppression and forcing act on when $\mathcal{M}$ rebuilds, poisoning targets what it rebuilds on. The mechanism is the benign-labelled injection of Section 5.5.2, but at a low rate that does not starve the gate. It instead rides the genuine mid-week drift event (the Wednesday arrival of the held-out class): when the real novel class arrives and the system fires, the mislabelled-benign flows the adversary has injected are themselves drifted, so they too are harvested into the concept-discovery buffer and carried into $\mathcal{M}$'s retraining set under the wrong label, intending to make $\mathcal{M}$ learn the novel attack pattern as benign and so inflate its per-class false-negative rate. Suppression and this poisoning are therefore two regimes of the same benign-labelled injection – at a high rate it starves the gate, at a low rate it survives the gate and becomes part of the harvested concept. So Chapter 6 sweeps the injection fraction across both. A durable variant injects only during the drift window and then stops, testing whether poison lodged in the never-evict exemplar memory (Section 5.4.3) persists across later clean rebuilds. How far $\mathcal{M}$ actually resists each is the subject of Section 6.4.

5.5.5 Backdooring the Base Learner

Poisoning that simply mislabels an attack as benign asks $\mathcal{M}$ to contradict all of the genuine, correctly labelled examples of that class present in the retraining set. A *backdoor* removes this problem: rather than moving the entire decision boundary, it installs a shortcut beside it. Injected flows are stamped with a small, fixed *trigger* – an additive perturbation on a handful of features – and are injected under a benign label, so that $\mathcal{M}$ learns the rule "attack features + trigger ⇒ benign". Formally, the trigger is a fixed perturbation τ supported on a small feature subset S (so $\tau_j = 0$ for $j \notin S$), and attack flows are injected as $(x + \tau, \textsc{Benign})$. Because $\mathcal{M}$ is an MLP over the raw flow (Section 5.3), the trigger acts directly on its input, independently of the encoder. The result is a *learned* backdoor rather than a mere evasion only if, after the rebuild,

$$\mathcal{M}(x+\tau) = \textsc{Benign} \quad \text{on a triggered attack } x, \qquad \text{while} \qquad \mathcal{M}_0(x+\tau) = \textsc{Attack}, \tag{5.8}$$

that is, the trigger flips the rebuilt learner while remaining inert on the un-poisoned learner $\mathcal{M}_0$ – a condition we verify directly in Section 6.4. As with the low-rate poisoning above, the trigger flows are drifted, so they are harvested into the concept buffer and into the never-evict exemplar memory, which replays them into every later rebuild and makes the backdoor persist even after the injection stops. At inference time genuine flows are stamped with the same trigger to evade $\mathcal{M}$ on demand. This realises the Backdoor $\mathcal{M}$ entry of Table 4.1.

5.6 Summary

This chapter turned the paper's specification into a runnable coupled system and formalised the attacks that probe it. Our reimplementation of the Contrastive NCM detector reproduces the paper's drift-detection behaviour ($F_1 = 0.887$ against 0.942; AUROC = 85.1 against 89.8, Section 5.2.2), validating it before any coupling is introduced. Deploying it then forced four refinements that complete components the paper leaves open: a count-and-fraction firing gate (at least 500 attack-labelled drifted samples making up at least 0.3 of the buffer), threshold calibration ($T_{drift} = 1.34$,

$T = 37.84$), an *accumulate, never-evict* rehearsal policy, and an evaluation surface that reports per-class false-negative rates alongside aggregate F_1. Three configurations – StaticDStaticM, StaticDAdaptiveM, and AdaptiveDAdaptiveM – form an ablation ladder that lets later experiments attribute effects to a specific mechanism. With these in place the clean system adapts correctly, reaching a Friday benchmark F_1 of 0.71 against a 0.207 no-adaptation floor: the reference point for every attack. Finally, all four evaluated attacks are instances of a single injection model parameterised by label and schedule, and Chapter 6 measures each against the channels of Chapter 4.

Chapter 6

Evaluation

This chapter evaluates the coupled adaptive system of Chapter 5 against the attacks specified in Section 4.6. It is organised around the three research questions of this thesis, relating to: the attack surface the coupling exposes (RQ1), poisoning attacks on the base learner through the composition channel (RQ2), and evasion attacks on the detector through the trigger channel (RQ3). Each is presented as a hypothesis, the results, and a discussion. Section 6.1 fixes the elements common to every experiment; experimental settings that vary between them are stated in a *Setup* note at the head of each section.

6.1 Experimental Settings

The victim throughout is the full coupled system, AdaptiveDAdaptiveM of Section 5.3.2, in which both the detector $\mathcal{D}$ and the base learner $\mathcal{M}$ are rebuilt at each concept-discovery event. Two reduced configurations serve as references where an ablation requires them: StaticDStaticM, which never adapts and fixes the no-adaptation floor, and StaticDAdaptiveM, in which the encoder is frozen but $\mathcal{M}$ still rebuilds.

The stream is the Monday–Friday CICIDS2017 trace (Engelen-corrected, Section 5.1) replayed in temporal order; `DoS` and `PortScan` are held out as the two novel classes that arrive mid-week, so that genuine drift is present in every run. The adversary acts through the injection model of Section 5.5.1. Every detector is calibrated by the rule of Section 5.4.2, but the resulting threshold values depend on the detector's training set and are therefore stated per experiment. We report F_1 measures against a fixed novel-class-aware benchmark and per-class false-negative rate (FNR) alongside it.

6.2 Baseline Behaviour Under Genuine Drift

Setup. The AdaptiveDAdaptiveM victim is trained on the full known-class pool ($T_{drift} = 1.34$, $T = 37.84$), run on the unpoisoned stream and averaged over three seeds.

Before introducing an adversary we confirm that, on an unpoisoned stream, the coupled system adapts to the genuine drift. It discovers `DoS` as it arrives mid-week (registering it as a new concept and rebuilding once) and recovers to a Friday F_1 of 0.71 against the no-adaptation floor of 0.207 (Table 6.1). It does *not*, however, discover `PortScan`: although its flows are flagged as drifted, they never make up enough of the buffer to clear the firing gate (Section 6.4), so the system adapts to only one of the two novel classes. This run is the reference point for every attack that follows.

Table 6.1 makes this concrete by comparing the full system (AdaptiveDAdaptiveM) to the static floor (StaticDStaticM, where $\mathcal{M}$ never updates), on aggregate F_1 and per-class false-negative rate. Both sit at the floor on Monday and Tuesday; once `DoS` is discovered during Wednesday, the full system's aggregate F_1 climbs from 0.207 to 0.710 and its `DoS` false-negative rate falls from 0.881 to 0.008, while the static floor stays put all week. However, the `PortScan` false-negative rate barely moves from the floor's 0.999 to 0.983. This is because the system never discovers that

class. We report per-class rates throughout for exactly this reason: aggregate F_1 alone would hide a whole novel class going undetected.

	StaticDStaticM			AdaptiveDAdaptiveM		
Day	F_1	FNR(`DoS`)	FNR(`PortScan`)	F_1	FNR(`DoS`)	FNR(`PortScan`)
Mon	0.207	0.881	0.999	0.207	0.881	0.999
Tue	0.207	0.881	0.999	0.207	0.881	0.999
Wed	0.207	0.881	0.999	0.346	0.640	0.995
Thu	0.207	0.881	0.999	0.710	0.008	0.983
Fri	0.207	0.881	0.999	0.710	0.008	0.983

Table 6.1: Baseline (no attack) per-day metrics for the static reference and the full system. F_1 is measured against a fixed novel-class-aware benchmark and false-negative rate is measured for `DoS` and `PortScan`, whose novel instances arrive on Wednesday and Friday respectively.

6.3 RQ1 – The Attack Surfaces of the Coupling

Hypothesis. The trigger-dependent coupling exposes attack surfaces that a continuously updated learner does not. Namely, a trigger channel that decides when rebuilds occur, and a composition channel that decides what data rebuilds use.

The baseline run is itself the evidence. Because $\mathcal{M}$ changes only when the predicate fires (Table 6.1), the trigger gates all adaptation. This means an adversary who controls whether the predicate fires controls whether $\mathcal{M}$ ever learns the emerging class, and an adversary who controls the buffer at a firing event controls what it learns. A continuously updated learner offers no such discrete gate; its decision boundary is a moving target that dilutes any single injection. The remainder of the chapter exploits exactly these two surfaces: RQ2 attacks the composition channel and RQ3 the trigger channel. Both are properties of the coupling and both are invisible to a detector-only evaluation.

6.4 RQ2 – The Composition Channel: Poisoning and Backdooring the Base Learner

Hypothesis. The composition channel (the data a firing event retrains on) lets the adversary corrupt $\mathcal{M}$ through its retraining set.

Setup. The victim is the full-data AdaptiveDAdaptiveM ($T_{drift} = 1.34$, $T = 37.84$). The adversary acts only through the benign-labelled injection of Section 5.5.1; its rate, schedule, and feature perturbations vary by experiment and are stated where each is introduced. All results average three seeds.

Contrary to the hypothesis, a base learner retrained from scratch to convergence *resists* this poisoning. Sweeping a benign-labelled injection of novel `DoS` flows over $p \in \{0, 5, \ldots, 35\}\%$ and riding the genuine mid-week drift event, so that the mislabelled flows are placed into $\mathcal{M}$'s retraining set, leaves the `DoS` false-negative rate essentially at its clean value across the low and mid budgets (0.008 at $p = 0$, 0.014 at 5%, 0.033 at 20% – Table 6.2). The mislabelled flows are a down-weighted minority in a set dominated by correctly labelled exemplars, so the rebuilt $\mathcal{M}$ still classifies `DoS` correctly. Persistence does not help: a *durable* variant that injects only around the fire and then stops, lodging its payload in the never-evict exemplar memory, is absorbed just the same. The large per-class damage that a single under-trained rebuild would expose is therefore an artefact of the retraining recipe, not of the coupling; an encoder-freeze ablation (StaticDAdaptiveM) confirms there is no representation-level poisoning to relocate, the frozen-encoder false-negative rate being no lower than the adapting one.

System	Poison %	Friday F_1	Fri FNR(`DoS`)	Fri FNR(`PS`)	Retrains
StaticDStaticM	0	0.207 ± 0.000	0.881 ± 0.000	0.999 ± 0.000	0
AdaptiveDAdaptiveM	0	0.710 ± 0.000	0.008 ± 0.000	0.983 ± 0.000	1
AdaptiveDAdaptiveM	5	0.726 ± 0.034	0.014 ± 0.006	0.903 ± 0.137	1.3±0.5
AdaptiveDAdaptiveM	15	0.820 ± 0.053	0.030 ± 0.003	0.529 ± 0.205	2.0±0.0
AdaptiveDAdaptiveM	20	0.821 ± 0.053	0.033 ± 0.002	0.508 ± 0.175	2.0±0.0
AdaptiveDAdaptiveM	25	0.207 ± 0.000	0.881 ± 0.000	0.999 ± 0.000	0.0±0.0
AdaptiveDAdaptiveM	30	0.680 ± 0.006	0.055 ± 0.011	1.000 ± 0.000	1.0±0.0
AdaptiveDAdaptiveM	35	0.702 ± 0.001	0.006 ± 0.001	1.000 ± 0.000	1.0±0.0

Table 6.2: Friday end-of-stream attack effect metrics on the full coupled system (AdaptiveDAdaptiveM). StaticDStaticM floor metrics under no attack are shown for reference. Values are mean ± std across three seeds.

So, the boundary cannot be moved against the genuine class, but it can be circumvented. The channel admits a backdoor, which succeeds precisely because it does not contend with the clean decision boundary. Rather than just mislabelling `DoS` as benign, the adversary also stamps them with a small feature-space trigger, teaching $\mathcal{M}$ that "`DoS`-features + trigger ⇒ benign". The two conditions for a learned backdoor (Eq. (5.8)) both hold: the trigger is inert on an un-poisoned learner – it moves the `DoS` false-negative rate from 0.012 to 0.014 – but after the backdoor is planted a *triggered* `DoS` flow evades $\mathcal{M}$ with false-negative rate 0.69, while *clean* `DoS` is still caught (with false-negative rate 0.03). The backdoor is also durable: the adversary injects only around the genuine mid-week fire and then stops, yet the backdoor persists regardless of any further retrains. This is because of all the harvested concept exemplars that carry the trigger being replayed into every subsequent rebuild – the never-evict exemplar memory (Section 5.4.3) turned against the system. An ablation that disables this memory isolates its role (Table 6.3). With the harvested exemplars retained (accum=500) the triggered false-negative rate stays consistently high (0.687 ± 0.096), whereas disabling them (accum=0) leaves the same mean (0.679) but makes persistence highly seed-dependent (±0.454). Therefore, we can confirm that the never-evict memory is what makes the backdoor reliable over time. That is, without the replayed exemplars, whether a later clean rebuild washes the shortcut out is left to chance. Here, the robust learner resists having its boundary moved, but not having a hidden shortcut installed beside it.

Condition	Friday F_1	FNR(Clean `DoS`)	FNR(Triggered `DoS`)	Retrains
Clean	0.787 ± 0.055	0.012 ± 0.005	0.014 ± 0.005	2.333 ± 1.247
Backdoor sustained	0.387 ± 0.254	0.588 ± 0.414	0.846 ± 0.109	0.333 ± 0.471
Backdoor (accum=500)	0.850 ± 0.051	0.034 ± 0.005	0.687 ± 0.096	2.333 ± 0.471
Backdoor (accum=0)	0.747 ± 0.041	0.014 ± 0.015	0.679 ± 0.454	1.333 ± 0.471

Table 6.3: Effects of backdooring the base learner through the composition channel. Benign-labelled `DoS` flows carrying a fixed feature-space trigger are injected (sustained, or poison-then-clean) and harvested into the retraining set. A backdoor is implanted when the false-negative rate on triggered `DoS` is high while clean `DoS` FNR and overall F_1 are unchanged. Values are mean ± std across three seeds and the accum=0 row disables the never-evict memory.

Beyond what the rebuilt $\mathcal{M}$ learns, the channel exposes control over whether it is rebuilt at all. Because the firing gate counts attack-labelled drifted samples (Section 5.4.1) and benign-labelled flows are drifted but not attack-labelled, a sufficient injection dilutes the buffer below the gate and starves retraining. At the knife-edge fraction $p = 25\%$ the system never fires for the whole stream, and F_1 collapses to the no-adaptation floor of 0.207 with both novel classes left unlearned. The collapse is sharp and non-monotonic ($p = 20\%$ and $p = 30\%$ still fire).

A final finding concerns the second novel class, and it concerns a property of our firing gate rather than of the detector. The system never discovers `PortScan`: its false-negative rate stays high (0.56–1.0) in every condition, attacked or not. This is *not* because the detector fails to flag it. Although only ∼2.5% of `PortScan` flows individually exceed T_{drift}, that is enough: an instrumented clean run shows that throughout the `PortScan` window the buffer holds over 1000 attack-labelled drifted flows (clearing the count floor of 500) and the accumulated displacement sits at $\|\Delta\| \geq 43.2 > T = 37.84$ (clearing the paper's displacement rule; Table 6.4). The only condition never met is our *fraction* floor. The buffer's attack fraction peaks at ∼0.20, below the 0.30 threshold, because the buffer is dominated by the detector's own benign false positives (calibrated at a ∼10% rate). In other words, `PortScan` is blind-spotted by exactly the dilution mechanism of the suppression attack above, occurring here with no adversary at all. It is therefore contingent on our calibration – a fraction floor of 0.20 rather than 0.30 would have admitted it. The blind spot is therefore a property of the makeup gate we add (Section 6.5.3), not a limitation of the detector's representation, and only a coupled, stream-side evaluation surfaces it.

Firing condition	Floor	`PortScan` window	Satisfied?
Displacement $\|\Delta\|$	$T = 37.84$	43.2–53.5	Yes (every batch)
Count (attack-labelled)	500	1008–3367	Yes (every batch)
Fraction	0.30	0.089–0.202	**No (never)**

Table 6.4: Firing-gate state during the `PortScan` window of a clean, unattacked run (batch-wise range over the Friday stream). `PortScan` clears the count and displacement conditions on every batch but never reaches the 0.3 fraction floor, so no concept-discovery event fires. The blind spot is therefore a property of the fraction floor, not of the detector's per-sample drift detection.

RQ2 resolves into a sharper claim than the hypothesis stated. The composition channel resists boundary poisoning, even when mislabelled flows end up in the never-evict memory they are simply absorbed. What the channel exposes instead is two things the robust learner here cannot defend against on its own: a backdoor that installs a trigger-activated shortcut beside the clean boundary and persists through the exemplar memory, and control over whether the system adapts at all (the $p = 25\%$ suppression collapse, compounded by the `PortScan` blind spot our own fraction floor creates with no adversary at all). Thus, the composition channel is exploitable, not by corrupting what $\mathcal{M}$ has learned, but by adding to it, or by deciding whether it learns (which we discuss further in Section 6.5).

6.5 RQ3 – The Trigger Channel: Forcing and Suppression Trade-Offs

Hypothesis. The detector can be forced to fire when it should not, or suppressed when it should, and the firing floors that resist one failure mode enable the other.

Setup. RQ3 spans two harnesses. The firing-gate ablation (Section 6.5.3) uses the full-data victim ($T = 37.84$). The transfer study (Sections 6.5.1 and 6.5.2) trains the victim on a stratified 75% of the known-class pool ($T = 39.29$) so that its grey-box surrogate can be trained on the disjoint 25%, sharing no training flows with the victim. The feature-space PGD uses an L_∞ budget swept over $\epsilon \in \{0.1, 0.25, 0.5, 1.0, 2.0\}$, white-box against the victim and grey-box across three independently seeded surrogates. All results are averaged over three seeds.

6.5.1 Forcing Per-Sample Detection

We separate two things that are easily conflated: making the detector *flag* a sample as drifted, and making the coupled system *fire* a concept-discovery event. A white-box PGD attack on the per-sample drift score saturates detection trivially – the flag rate on perturbed known-class flows rises from a clean 0.10 to 1.0 by $\epsilon = 0.5$ (Fig. 6.1). But transfer to grey-box is weak; crafted on a data-disjoint surrogate, the same perturbations move the victim's flag rate to only ~ 0.32 at $\epsilon = 0.5$, rising to at most 0.60 at $\epsilon = 2.0$. Furthermore, saturating per-sample detection does not necessarily force a fire. Injected in a stable pre-drift window these flows leave the concept-discovery accumulator below T and trigger no events (for both white-box and grey-box), because firing depends on the displacement of the drifted-batch mean, not on individual flags.

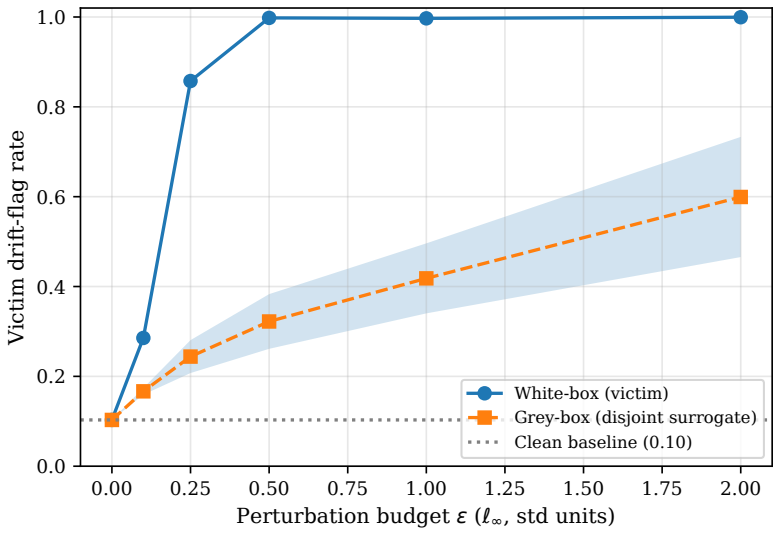

Figure 6.1: PGD false-positive attack on per-sample detection: fraction of perturbed known-class flows the victim flags as drifted versus the L_∞ budget, white-box against the victim and grey-box via surrogate transfer.

6.5.2 Forcing the Concept Trigger

Forcing therefore requires displacing the batch mean coherently. Naively applying coherent perturbations along a random direction fails. Even white-box at $\epsilon = 2.0$ the accumulated displacement reaches only 29.5, short of the required threshold $T = 39.3$, and forces no fire. This is what initially looked like robustness. However, we conclude that it is not; it is a weak attack. An aimed mean-shift attack, which optimises two opposing perturbations to maximise the separation of the encoder's batch means (Eq. (5.7)), crosses T at $\epsilon \geq 1.0$, reaching displacements of 47 at $\epsilon = 1.0$ and 92 at $\epsilon = 2.0$ (Table 6.5), thereby forcing a concept-discovery event white-box.

L_∞ budget (ϵ)	Flag A	Flag B	Displacement	Threshold (T)	Forces fire?
0.10	0.23	0.11	18.0	39.3	False
0.25	0.75	0.17	26.1	39.3	False
0.50	0.77	0.86	25.6	39.3	False
1.00	1.00	1.00	47.1	39.3	True
2.00	1.00	1.00	91.5	39.3	True

Table 6.5: White-box accumulator attack via direct latent mean-shift. PGD jointly optimises two pools to maximise the separation of their drifted-batch means (the quantity the concept-discovery accumulator integrates), rather than pushing along a fixed random direction. A fire is forced only if the displacement exceeds T.

The most consequential result is that this attack transfers. Crafted against a data-disjoint surrogate and evaluated on the victim, the mean-shift retains roughly 70% of its white-box displacement – 33 at $\epsilon = 1.0$ and 65 at $\epsilon = 2.0$ (Table 6.6). That clears $T = 39.3$ and forces a fire at $\epsilon = 2.0$ while at $\epsilon = 1.0$ the transferred displacement falls just short. This stands in sharp contrast to per-sample detection, which transferred poorly ($\leq 60\%$): a per-sample flag requires every sample to individually clear the threshold, which is brittle across two independently trained encoders. Whereas, the trigger fires on a batch mean and averaging launders the per-sample transfer noise, leaving the coherent direction both encoders agree on. The batch-mean statistic the detector adopts for stability is precisely what makes the trigger grey-box forceable.

L_∞ budget (ϵ)	Displacement	Threshold (T)	Seeds forcing fire
0.10	11.1 ± 0.7	39.3	0/3
0.25	18.3 ± 1.1	39.3	0/3
0.50	20.7 ± 1.6	39.3	0/3
1.00	33.1 ± 2.0	39.3	0/3
2.00	65.0 ± 3.8	39.3	3/3

Table 6.6: Grey-box mean-shift transfer. Two pools are optimised against a disjoint surrogate (trained on a held-out 25% of the known-class pool, with no flows shared with the victim) and the displacement of their drifted-batch means is measured on the victim. Reported values are mean $\pm$ std of the transferred displacement over three surrogate seeds and the number of seeds whose transfer forces a victim concept-discovery fire. A fire requires the displacement to exceed T (compare the white-box mean-shift, Table 6.5).

The caveat is the budget. Grey-box forcing requires $\epsilon = 2.0$ here, a large, unconstrained L_∞ perturbation in standardised-feature units that would almost certainly distort flows beyond what protocol constraints permit; a feature-constrained mean-shift would raise the bar further and is left to future work. Within the adopted grey-box threat model, however, the conclusion stands: the concept trigger is forceable, not robust, and the attack is bounded only by the perturbation budget.

6.5.3 Decomposing the Firing Gate

The firing predicate is made up of three conjuncts – displacement, count, and fraction (Eq. (5.1)). To attribute the system's behaviour to the individual floors we ablate the count and fraction conditions independently, across `displacement-only` (the paper's rule, neither floor), `count-only`, `fraction-only`, and the `full` gate we propose.

Gate	Poison %	Forced fires
Displacement-only	2%	0.667 ± 0.943
	5%	14.667 ± 2.867
	10%	5.000 ± 2.160
	20%	0.000 ± 0.000
Count-only	2%	0.333 ± 0.471
	5%	1.667 ± 0.943
	10%	2.333 ± 0.943
	20%	0.000 ± 0.000
Fraction-only	2%	0.000 ± 0.000
	5%	0.000 ± 0.000
	10%	0.000 ± 0.000
	20%	0.000 ± 0.000
Full	2%	0.000 ± 0.000
	5%	0.000 ± 0.000
	10%	0.000 ± 0.000
	20%	0.000 ± 0.000

Table 6.7: Gate ablation (forcing): pre-drift injection of true-labelled novel flows under the four decomposed firing gates of the count/fraction 2x2. Values reported are mean ± std over seeds.

Gate	Fri F_1	Retrains
Displacement-only	0.620 ± 0.036	134.667 ± 22.366
Count-only	0.207 ± 0.000	0.000 ± 0.000
Fraction-only	0.207 ± 0.000	0.000 ± 0.000
Full	0.207 ± 0.000	0.000 ± 0.000

Table 6.8: Gate ablation (suppression): benign-labelled injection at p=25% during genuine drift under the four decomposed firing gates of the count/fraction 2x2. Reported values are the mean ± std over seeds.

Against forcing (Table 6.7, pre-drift injection of true-labelled flows), the two floors are not equal. The fraction floor alone blocks every forced fire at every budget, because at low injection the attacker's flows are only a small fraction of the drifted buffer which is already padded by the detector's own false positives, so the 0.3 threshold is never reached. The count floor reduces the efficacy of forcing. It cuts forced fires from 14.7 to 1.7 at $p = 5\%$, but it alone is not sufficient since the 500 samples it demands are within a determined attacker's budget. The full gate, exactly like fraction-only, is completely immune.

Against suppression (Table 6.8, benign injection at $p = 25\%$), the picture inverts. Either floor alone suffices to make the system suppressible. The `count-only`, `fraction-only`, and `full` gates all collapse to the no-adaptation floor ($F_1 = 0.207$, zero retrains), while only `displacement-only` keeps adapting. However, it does so pathologically, firing approximately 135 times on the injected noise, and is in turn the most forceable gate of all. The only configuration we examine that resists suppression is the one that cannot resist forcing.

The ablation demonstrates the trade-off of this trigger-gated adaptation mechanism. The suppression vector is over-determined. Any floor placed on the buffer's makeup to resist forcing or premature firing simultaneously hands the adversary a way to hold the buffer below it and starve adaptation. The two floors are defeated by different mechanisms – the fraction floor by direct dilution (benign flows drive the attack fraction below 0.3), the count floor is more subtle. Because the displacement accumulator is only reset when the trigger is fired (Eq. (2.17)), a gate that withholds firing lets it telescope into a stationary regime in which $\|\Delta\|$ stays below T even after the count is satisfied. Resetting or windowing the accumulator on gated crossings could be implemented as

mitigations, but we leave this to future work. The design implication is stark: under makeup-based gating, making the trigger harder to force and harder to suppress are in direct tension.

This tension does not depend on the particular values we chose. Resisting forcing *requires* the fraction floor specifically – the count floor alone leaks (Table 6.7) – and any fraction floor, whatever its threshold, is suppressible by dilution, because the adversary controls the buffer's makeup in both directions: raising the attack fraction to force a fire and lowering it to starve one. The 0.3 value sets only the budget each attack needs, not whether the vulnerability exists. We demonstrate this at a single operating point rather than sweeping the floors. Tellingly, the same dilution prevents detection of a genuine novel class with no adversary at all (the `PortScan` case of Section 6.4). A sweep mapping the full force-versus-suppress trade-off, and whether any calibration balances the two, is left to future work (Section 7.2).

6.6 Summary

The three research questions resolve as follows. RQ1: the trigger-dependent coupling exposes a trigger channel and a composition channel that gate all adaptation, and both are invisible to the detector-only evaluation the literature uses. RQ2: the composition channel *resists* boundary poisoning but it does *not* resist a backdoor, which installs a trigger-activated shortcut beside the clean boundary and persists through the exemplar memory; the same channel also decides whether $\mathcal{M}$ adapts at all (the $p = 25\%$ suppression collapse), compounded by a blind spot in which our fraction floor, diluted by the detector's own false positives, never admits `PortScan`. RQ3: the firing trigger is forceable, not robust. An aimed mean-shift attack forces a concept-discovery event both white-box and grey-box (retaining roughly 70% of its displacement against a data-disjoint surrogate); and the gate ablation shows the two firing floors are over-determined, so any floor that resists forcing simultaneously enables suppression. Across all three, the throughline is that coupling a robust detector to a base learner does not make the system robust: the security is further decided at the coupling, a conclusion Chapter 7 draws together.

Chapter 7

Conclusion

We set out to evaluate Kuppa and Le-Khac's Contrastive NCM drift detector [4] not in isolation, as the literature does, but coupled to the downstream base learner it is meant to supervise – the configuration in which it would actually be deployed. That coupling, not the detector's representation, is where the security of the system is decided.

Our central finding is that the coupling exposes the adversary to the system's adaptation rather than to its learned decision boundary directly. That is, whether and when the base learner is rebuilt, and what is added to it at a rebuild. Three results make the case. First (RQ1), because the base learner changes only at discrete concept-discovery events, an adversary who controls the firing predicate, or the buffer at a fire, controls what the learner learns and if it learns at all; a continuously updated learner offers no such gate. Second (RQ3), the firing trigger is *forceable and not robust*: an aimed feature-space mean-shift forces a concept-discovery event both white-box and grey-box, retaining roughly 70% of its displacement against a data-disjoint surrogate, while the per-sample detection it is built on transfers poorly. Thus, it is the batch-mean statistic the detector adopts for stability that makes the trigger transferable. The firing floors that resist forcing are themselves double-edged: the gate ablation proves that any floor on the buffer's makeup which blocks forcing simultaneously enables suppression, an over-determined trade-off intrinsic to makeup-based gating. Third (RQ2), the composition channel resists poisoning that contradicts $\mathcal{M}$'s decision boundary: a from-scratch learner holds its `DoS` false-negative rate near its clean 0.008 across injection budgets. However, it does *not* resist a backdoor: a benign-labelled, trigger-stamped payload installs a shortcut that evades $\mathcal{M}$ on demand (a false-negative rate of 0.69 on triggered traffic against 0.03 on clean), leaves aggregate accuracy intact, and persists via the never-evict exemplar memory. We also find that the same channel governs whether the learner adapts at all. A benign dilution of $p = 25\%$ holds the gate shut for the whole stream and collapses the system to the no-adaptation floor, F_1 falling from 0.71 to 0.207. Finally, the system never discovers the second novel class – but not because the detector fails to flag it. `PortScan` clears both the count and displacement conditions; it is blocked only by our 0.30 fraction floor, diluted below threshold by the detector's own benign false positives. The blind spot is thus the suppression mechanism arising with no adversary at all, an artefact of our gate's calibration rather than of the detector.

A key takeaway is that coupling a robust detector to a base learner does not guarantee that the system will inherit the detector's robustness; it creates new attack surfaces that the detector-only evaluation the original proposal invites cannot see. A drift detector that is robust as a classifier can still be an exploitable *trigger* or *supervisor*.

Several broader lessons follow. The first is that *robustness does not compose*: a component certified robust in isolation can be redeployed as a trigger or a supervisor whose failure modes its own evaluation never measured, so security has to be assessed at the level of the coupled system rather than the component. The second is that the design choices made for good behaviour are exactly what the adversary turns against the system – the batch-mean statistic adopted to make the trigger *stable* is what makes it transferable, and the rehearsal memory adopted to prevent catastrophic *forgetting* is what makes a backdoor durable. The third is that a defended decision boundary is not a defended system: with the boundary held, the adversary's leverage simply moves to whether

and exactly when the learner is rebuilt, and to whichever class the detector never learns to flag. Finally, the experiments are a reminder that aggregate metrics conceal precisely the damage that matters here; per-class reporting was necessary to see a whole attack class go undetected behind an almost-unchanged F_1. Each is a factor that designers of coupled adaptive systems must weigh, and each is invisible to the detector-only evaluation the current research relies on.

7.1 Future Work

Several threads follow directly. A *feature-constrained* mean-shift, restricting perturbations to *protocol-valid* flows, would establish how forceable the trigger is for a realistic attacker, since our attack succeeds only at an unconstrained L_∞ budget. The backdoor we demonstrate uses a synthetic feature trigger; realising a protocol-valid trigger on genuine flows and defending against it, for instance by sanitising the harvested exemplars before they are replayed, is the natural follow-up. The *blind spot* invites a *stealth-novelty* attack: since a novel class is admitted only once its flows make up enough of the drifted buffer, an adversary could pace a novel attack under benign cover so that its attack fraction never reaches the gate's fraction floor, keeping the coupled system from ever adapting to it. Finally, from a defensive perspective, resetting or windowing the displacement accumulator on gated crossings would likely close the count-floor suppression mechanism, and a continuously updated base-learner baseline would quantify what the discrete trigger costs in robustness.

7.2 Limitations

The findings of Chapter 6 are reported at a single calibrated operating point. The per-sample drift threshold T_{drift} is fixed to a 10% known-class false-positive rate and the concept threshold T to the 0.995 displacement quantile (Section 5.4.2); both are principled choices, but the attack results are necessarily conditional on them. Because Kuppa and Le-Khac underspecify their own operating point, we could not validate these thresholds by reproduction (which is why the detector itself is validated threshold-free, on AUROC, in Section 5.2.2); we validate them instead by the system's correct adaptation under genuine drift. A systematic sensitivity analysis – sweeping T_{drift} across false-positive rates and T across quantiles – would establish how far the findings depend on this calibration, and is left to future work.

The firing floors (count 500, fraction 0.3) are likewise calibrated rather than optimised: the fraction floor is set above the detector's $\sim$10% false-positive contamination of the buffer and the count floor at a minimum viable sample for a stable prototype (Section 5.4.1). Our claims are scoped accordingly. The over-determination of the makeup gate is argued structurally and so does not depend on these values (Section 6.5.3), whereas the magnitude of each attack's budget, and the `PortScan` blind spot, are conditional on this operating point. A sweep of the floors would map the force-versus-suppress trade-off curve. However, because our contribution is to characterise the attack surface rather than to optimise a defence, we leave that, and the choice of an operating point that best balances the two, to future work.

Bibliography

[1] Gao J, Fan W, Han J, Yu PS. A General Framework for Mining Concept-Drifting Data Streams with Skewed Distributions. In: Proceedings of the 2007 SIAM International Conference on Data Mining. SIAM; 2007. p. 3-14. Available from: https://doi.org/10.1137/1.9781611972771.1.

[2] Gama J, Žliobaitė I, Bifet A, Pechenizkiy M, Bouchachia A. A survey on concept drift adaptation. ACM Computing Surveys. 2014 Mar;46(4):1-37. Available from: https://doi.org/10.1145/2523813.

[3] Barros RSM, Santos SGTC. A large-scale comparison of concept drift detectors. Information Sciences. 2018 Jul;451–452:348-70. Available from: https://doi.org/10.1016/j.ins.2018.04.014.

[4] Kuppa A, Le-Khac NA. Learn to adapt: Robust drift detection in security domain. Computers and Electrical Engineering. 2022 Sep;102:108239. Available from: https://doi.org/10.1016/j.compeleceng.2022.108239.

[5] Korycki Ł, Krawczyk B. Adversarial concept drift detection under poisoning attacks for robust data stream mining. Machine Learning. 2022 Jun;112(10):4013-48. Available from: https://doi.org/10.1007/s10994-022-06177-w.

[6] Lu J, Liu A, Dong F, Gu F, Gama J, Zhang G. Learning under Concept Drift: A Review. IEEE Transactions on Knowledge and Data Engineering. 2018 Oct;31(12):2346-63. Available from: https://doi.org/10.1109/TKDE.2018.2876857.

[7] Guzy F, Woźniak M. Employing dropout regularization to classify recurring drifted data streams. In: 2020 International Joint Conference on Neural Networks (IJCNN). IEEE; 2020. p. 1-7. Available from: https://doi.org/10.1109/IJCNN48605.2020.9207266.

[8] Gama J, Medas P, Castillo G, Rodrigues P. Learning with Drift Detection. In: Bazzan ALC, Labidi S, editors. Advances in Artificial Intelligence – SBIA 2004. Springer Berlin Heidelberg; 2004. p. 286-95. Available from: https://doi.org/10.1007/978-3-540-28645-5_29.

[9] Bifet A, Gavaldà R. Learning from Time-Changing Data with Adaptive Windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining. SIAM; 2007. p. 443-8. Available from: https://doi.org/10.1137/1.9781611972771.42.

[10] Song X, Wu M, Jermaine C, Ranka S. Statistical change detection for multi-dimensional data. In: Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD07. ACM; 2007. p. 667-76. Available from: https://doi.org/10.1145/1281192.1281264.

[11] Qahtan AA, Alharbi B, Wang S, Zhang X. A PCA-Based Change Detection Framework for Multidimensional Data Streams: Change Detection in Multidimensional Data Streams. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '15. ACM; 2015. p. 935-44. Available from: https://doi.org/10.1145/2783258.2783359.

[12] Kloft M, Laskov P. Online Anomaly Detection under Adversarial Impact. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. vol. 9. PMLR; 2010. p. 405-12. Available from: https://proceedings.mlr.press/v9/kloft10a.html.

[13] Sethi TS, Kantardzic M. Handling adversarial concept drift in streaming data. Expert Systems with Applications. 2018 May;97:18-40. Available from: https://doi.org/10.1016/j.eswa.2017.12.022.

[14] French RM. Catastrophic forgetting in connectionist networks. Trends in Cognitive Sciences. 1999 Apr;3(4):128-35. Available from: https://doi.org/10.1016/S1364-6613(99)01294-2.

[15] Robins A. Catastrophic Forgetting, Rehearsal and Pseudorehearsal. Connection Science. 1995 Jun;7(2):123-46. Available from: https://doi.org/10.1080/09540099550039318.

[16] Rebuffi SA, Kolesnikov A, Sperl G, Lampert CH. iCaRL: Incremental Classifier and Representation Learning. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE; 2017. p. 5533-42. Available from: https://doi.org/10.1109/CVPR.2017.587.

[17] Sharafaldin I, Lashkari AH, Ghorbani AA. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). SciTePress; 2018. p. 108-16. Available from: https://doi.org/10.5220/0006639801080116.

[18] Engelen G, Rimmer V, Joosen W. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In: 2021 IEEE Security and Privacy Workshops (SPW). IEEE; 2021. p. 7-12. Available from: https://doi.org/10.1109/SPW53761.2021.00009.

[19] Liu L, Engelen G, Lynar T, Essam D, Joosen W. Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In: 2022 IEEE Conference on Communications and Network Security (CNS). IEEE; 2022. p. 254-62. Available from: https://doi.org/10.1109/CNS56114.2022.9947235.

[20] Kuppa A, Grzonkowski S, Asghar MR, Le-Khac NA. Black Box Attacks on Deep Anomaly Detectors. In: Proceedings of the 14th International Conference on Availability, Reliability and Security (ARES). ACM; 2019. p. 1-10. Available from: https://doi.org/10.1145/3339252.3339266.

Chapter 8

Declarations

8.1 Use of Generative AI

I acknowledge the use of Claude and Claude Code (Anthropic, https://claude.ai/) and the underlying models (Opus 4.8 and Sonnet 4.6) while undertaking this work. I wrote the reimplementation of the Contrastive NCM detector that this thesis evaluates and all of the attack implementations. AI was used to write supporting code that is not part of the thesis's contribution: the multi-layer-perceptron base-learner module, the CICIDS2017 data-loading and preprocessing utilities, and the figure- and table-generation helpers. Under my own design and supervision, it also assisted with implementing the streaming harness and I used it for boilerplate in the experiment notebooks and for debugging and refactoring throughout. For the report itself, the content, findings, and contributions (the threat model, the attack taxonomy, the experimental design, and the analysis and conclusions) are my own. I primarily used AI for proofreading, refining language, and structured feedback on drafts. All AI-generated code was read, tested, and integrated by me, and I am responsible for its correctness. No AI output has been presented as my own original contribution or relied upon as a factual source without independent verification.

8.2 Ethical Considerations

This work takes an adversarial perspective on a deployed adaptive intrusion-detection architecture, developing concrete attacks that force or suppress its adaptation and poison or backdoor it through its retraining set [4]. Therefore, it falls under *offensive security* or *red teaming*. Documenting such attacks carries the inherent risk of providing a blueprint for adversaries, but "security through obscurity" is a fragile defence, and it is preferable for these weaknesses to be identified and documented here than discovered by real-world attackers. To that end, every attack is reported alongside the mechanism that enables it (Chapters 6 and 7).

All experiments are conducted entirely within a sandboxed simulation, on local or authorised university hardware, using only the publicly available CICIDS2017 benchmark. No live systems, third-party services, or commercial APIs are probed or targeted, and no personally identifiable information is involved. The project is conducted in accordance with the *Computer Misuse Act 1990 (UK)*.

8.3 Sustainability

The computational footprint of this work is modest and was actively managed. Experiments were run on a single NVIDIA RTX 3090 GPU (a rented `vast.ai` instance) and on an Imperial-provided virtual machine with a Tesla T4 GPU. The dominant cost is the repeated retraining of the detector's encoder and the base learner at each concept-discovery event. This is bounded by the relatively small models involved and by fixed, short training schedules rather than open-ended optimisation. The streaming harness keeps data GPU-resident to avoid repeated host–device transfers, and design choices were fixed from a small number of targeted runs rather than exhaustive hyperparameter

searches. Relative to contemporary deep-learning workloads the datasets and models used here are small, so the overall energy use and carbon footprint of the project are correspondingly low.

8.4 Availability of Data and Materials

This work relies exclusively on publicly available data. The evaluation uses the CICIDS2017 intrusion-detection benchmark [17] in its Engelen-corrected release [18], both of which are openly available to researchers. No proprietary or personally identifiable information was collected, processed, or generated, so the risk of privacy harm is negligible. Because Kuppa and Le-Khac [4] publish no reference implementation, the Contrastive NCM detector, the coupled adaptive system, and all attacks were reimplemented from scratch for this project. The full source code, together with the experiment notebooks, is available at https://github.com/dillan-scott/final-year-project-dbs21.